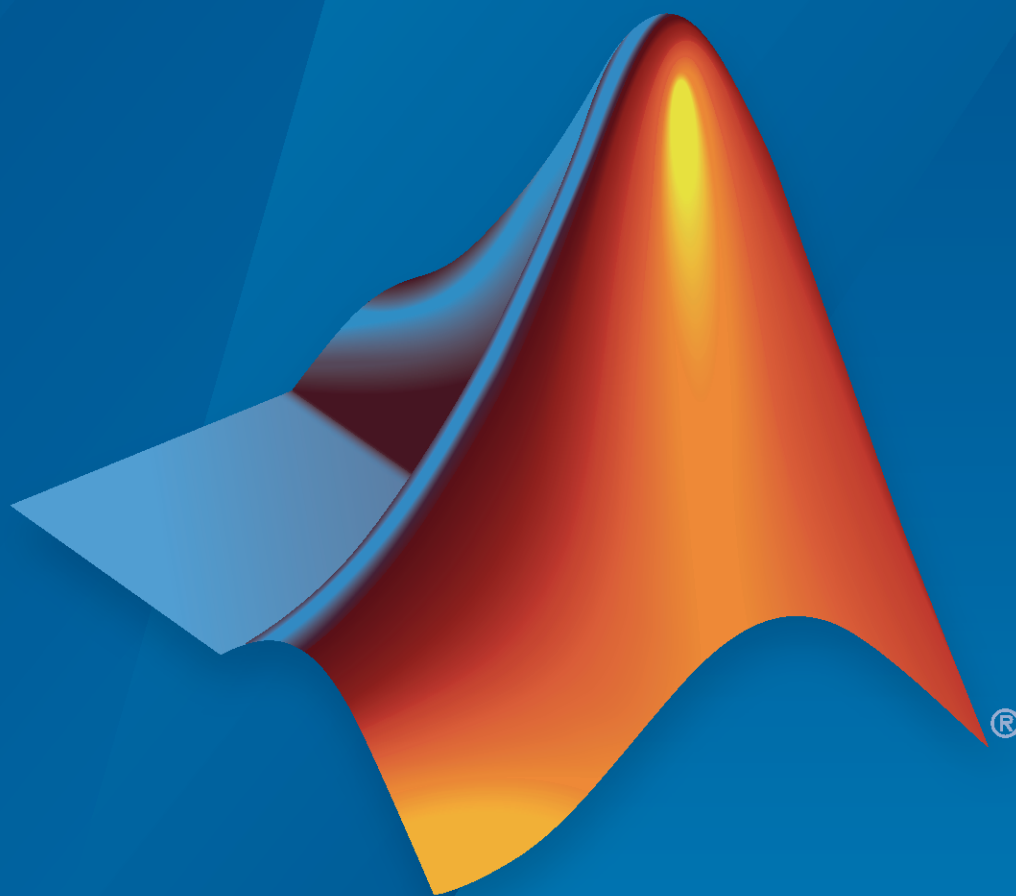


Simulink®

Getting Started Guide



MATLAB® & SIMULINK®

R2026b



How to Contact MathWorks



Latest news: www.mathworks.com

Sales and services: www.mathworks.com/sales_and_services

User community: www.mathworks.com/matlabcentral

Technical support: www.mathworks.com/support/contact_us



Phone: 508-647-7000



The MathWorks, Inc.
1 Apple Hill Drive
Natick, MA 01760-2098

Simulink® Getting Started Guide

© COPYRIGHT 1990-2026 by The MathWorks, Inc.

The software described in this document is furnished under a license agreement. The software may be used or copied only under the terms of the license agreement. No part of this manual may be photocopied or reproduced in any form without prior written consent from The MathWorks, Inc.

FEDERAL ACQUISITION: This provision applies to all acquisitions of the Program and Documentation by, for, or through the federal government of the United States. By accepting delivery of the Program or Documentation, the government hereby agrees that this software or documentation qualifies as commercial computer software or commercial computer software documentation as such terms are used or defined in FAR 12.212, DFARS Part 227.72, and DFARS 252.227-7014. Accordingly, the terms and conditions of this Agreement and only those rights specified in this Agreement, shall pertain to and govern the use, modification, reproduction, release, performance, display, and disclosure of the Program and Documentation by the federal government (or other entity acquiring for or through the federal government) and shall supersede any conflicting contractual terms or conditions. If this License fails to meet the government's needs or is inconsistent in any respect with federal procurement law, the government agrees to return the Program and Documentation, unused, to The MathWorks, Inc.

Trademarks

MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See www.mathworks.com/trademarks for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

Patents

MathWorks products are protected by one or more U.S. patents. Please see www.mathworks.com/patents for more information.

Revision History

September 2005	Online only	New for Version 6.3 (Release 14SP3)
March 2006	Online only	Revised for Simulink 6.4 (Release 2006a)
September 2006	Online only	Revised for Simulink 6.5 (Release 2006b)
March 2007	First printing	Revised for Simulink 6.6 (Release 2007a)
September 2007	Second printing	Revised for Simulink 7.0 (Release 2007b)
March 2008	Third printing	Revised for Simulink 7.1 (Release 2008a)
October 2008	Fourth printing	Revised for Simulink 7.2 (Release 2008b)
March 2009	Fifth printing	Revised for Simulink 7.3 (Release 2009a)
September 2009	Online only	Revised for Simulink 7.4 (Release 2009b)
March 2010	Online only	Revised for Simulink 7.5 (Release 2010a)
September 2010	Online only	Revised for Simulink 7.6 (Release 2010b)
April 2011	Online only	Revised for Simulink 7.7 (Release 2011a)
September 2011	Sixth printing	Revised for Simulink 7.8 (Release 2011b)
March 2012	Seventh printing	Revised for Simulink 7.9 (Release 2012a)
September 2012	Eighth printing	Revised for Simulink 8.0 (Release 2012b)
March 2013	Ninth printing	Revised for Simulink 8.1 (Release 2013a)
September 2013	Tenth printing	Revised for Simulink 8.2 (Release 2013b)
March 2014	Eleventh printing	Revised for Simulink 8.3 (Release 2014a)
October 2014	Twelfth printing	Revised for Simulink 8.4 (Release 2014b)
March 2015	Thirteenth printing	Revised for Simulink 8.5 (Release 2015a)
September 2015	Fourteenth printing	Revised for Simulink 8.6 (Release 2015b)
October 2015	Online only	Rereleased for Simulink 8.5.1 (Release 2015aSP1)
March 2016	Fifteenth printing	Revised for Simulink 8.7 (Release 2016a)
September 2016	Sixteenth printing	Revised for Simulink 8.8 (Release 2016b)
March 2017	Seventeenth printing	Revised for Simulink 8.9 (Release 2017a)
September 2017	Eighteenth printing	Revised for Simulink 9.0 (Release 2017b)
March 2018	Nineteenth printing	Revised for Simulink 9.1 (Release 2018a)
September 2018	Twentieth printing	Revised for Simulink 9.2 (Release 2018b)
March 2019	Online only	Revised for Simulink 9.3 (Release 2019a)
September 2019	Online only	Revised for Simulink 10.0 (Release 2019b)
March 2020	Online only	Revised for Simulink 10.1 (Release 2020a)
September 2020	Online only	Revised for Simulink 10.2 (Release 2020b)
March 2021	Online only	Revised for Simulink 10.3 (Release 2021a)
September 2021	Online only	Revised for Simulink 10.4 (Release 2021b)
March 2022	Online only	Revised for Simulink 10.5 (Release 2022a)
September 2022	Online only	Revised for Simulink 10.6 (Release 2022b)
March 2023	Online only	Revised for Simulink 10.7 (Release 2023a)
September 2023	Online only	Revised for Version 23.2 (R2023b)
March 2024	Online only	Revised for Version 24.1 (R2024a)
September 2024	Online only	Revised for Version 24.2 (R2024b)
March 2025	Online only	Revised for Version 25.1 (R2025a)
September 2025	Online only	Revised for Version 25.2 (R2025b)
March 2026	Online only	Revised for Version 26.1 (R2026a)
September 2026	Online only	Revised for Version 26.2 (R2026b)

1	Introduction	
	Simulink Product Description	1-2
	Model-Based Design with Simulink	1-3
	Example Model-Based Design Workflow in Simulink	1-4
	System Definition and Layout	1-7
	Determine Modeling Objectives	1-7
	Identify System Components and Interfaces	1-8
	Model and Validate a System	1-13
	Open System Layout	1-13
	Model Physical Components	1-14
	Model Functional Components	1-17
	Set Model Parameters	1-17
	Validate Components	1-18
	Validate Model	1-20
	Design a System in Simulink	1-25
	Open System Model	1-25
	Identify Designed Components and Design Goals	1-25
	Analyze System Behavior Using Simulation	1-26
	Design Components and Verify Design	1-28
2	Modeling in Simulink	
	Simulink Block Diagrams	2-2
3	Simple Simulink Model	
	Create a Simple Model	3-2
	Open New Model	3-3
	Open Simulink Library Browser	3-5
	Add Blocks to Model	3-7
	Connect Blocks	3-9
	Edit Block Parameter Values	3-11
	Run Simulation	3-12

View Simulation Data	3-12
Refine Model	3-13
Provide Feedback on the Software	3-19

4

Navigate a Simulink Model

Explore Model Hierarchy	4-2
View Model Hierarchy	4-2
View Signal Attributes	4-4
Trace a Signal	4-5

Introduction

- “Simulink Product Description” on page 1-2
- “Model-Based Design with Simulink” on page 1-3
- “System Definition and Layout” on page 1-7
- “Model and Validate a System” on page 1-13
- “Design a System in Simulink” on page 1-25

Simulink Product Description

Simulation and Model-Based Design

Simulink is a block diagram environment for multidomain simulation and Model-Based Design. It supports system-level design, simulation, automatic code generation, and continuous test and verification of embedded systems. Simulink provides a graphical editor, customizable block libraries, and solvers for modeling and simulating dynamic systems. It is integrated with MATLAB®, enabling you to incorporate MATLAB algorithms into models and export simulation results to MATLAB for further analysis.

Model-Based Design with Simulink

Modeling is a way to create a virtual representation of a real-world system. You can simulate this virtual representation under a wide range of conditions to see how it behaves.

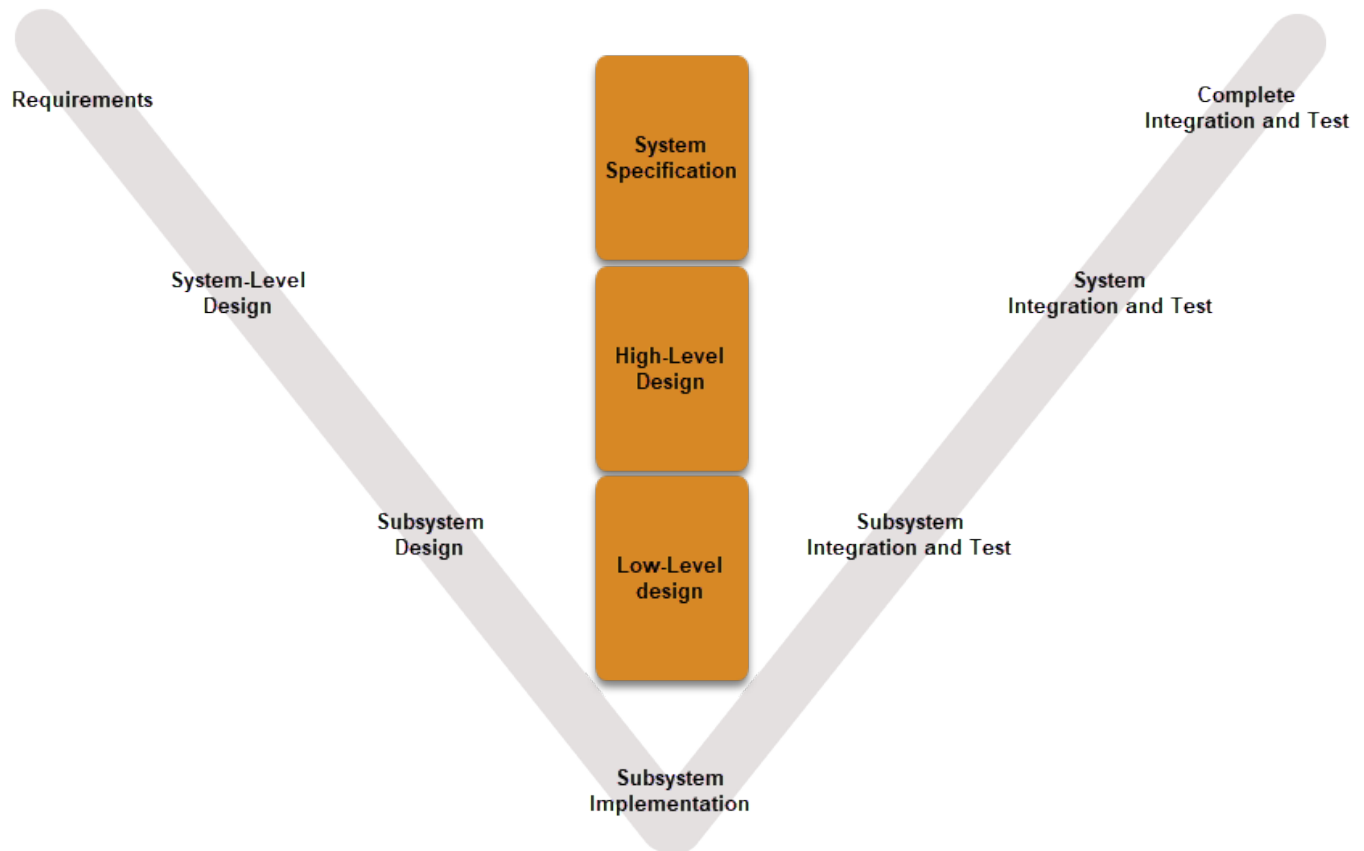
Modeling and simulation are valuable for testing conditions that are difficult to reproduce with hardware prototypes alone. This is especially true in the early phase of the design process when hardware is not yet available. Iterating between modeling and simulation can improve the quality of the system design early, by reducing the number of errors found later in the design process.

You can automatically generate code from a model and, when software and hardware implementation requirements are included, create test benches for system verification. Code generation saves time and prevents the introduction of manually coded errors.

In Model-Based Design, a system model is at the center of the workflow. Model-Based Design enables fast and cost-effective development of dynamic systems, including control systems, signal processing systems, and communications systems.

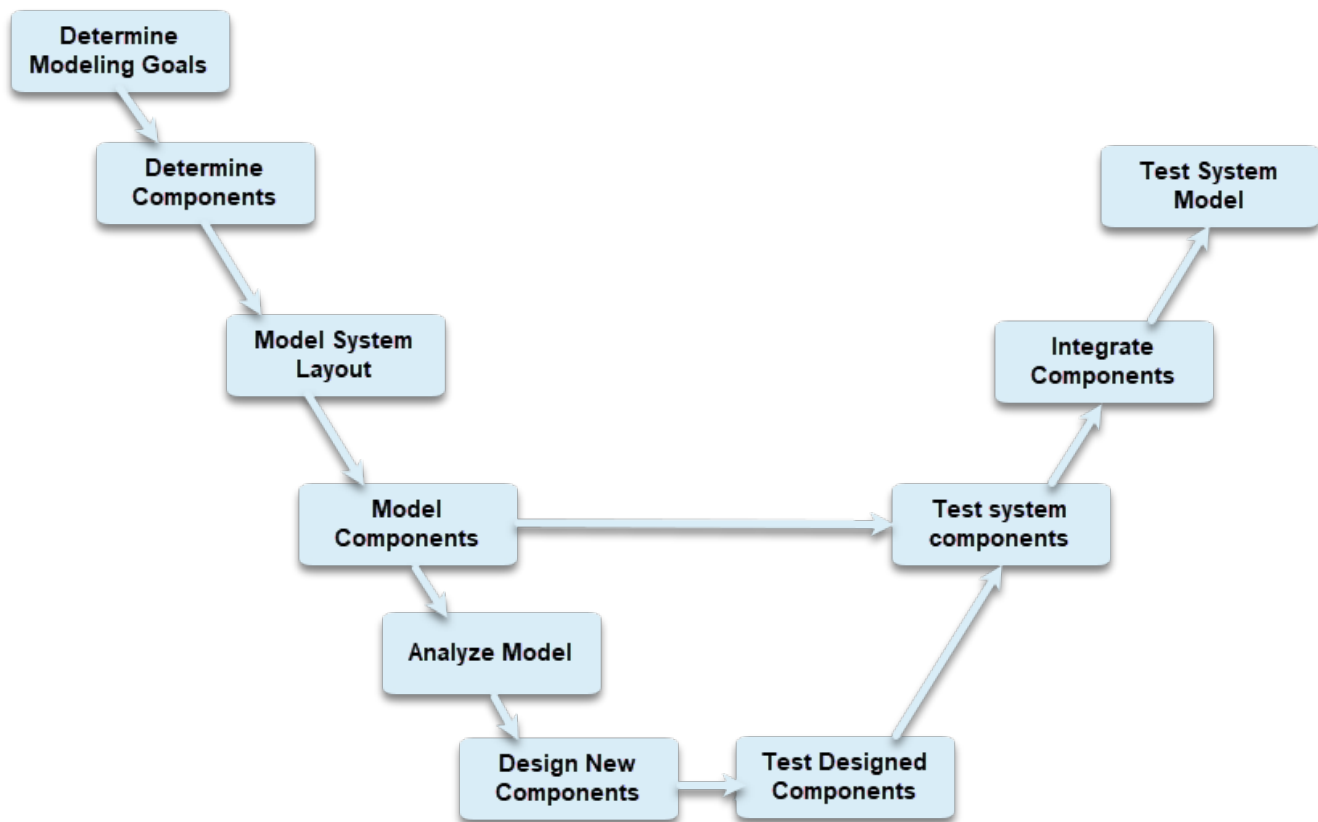
Model-Based Design allows you to:

- Use a common design environment across project teams
- Link designs directly to requirements
- Identify and correct errors continuously by integrating testing with design
- Refine algorithms through multidomain simulation
- Automatically generate embedded software code and documentation
- Develop and reuse test suites



Example Model-Based Design Workflow in Simulink

To get started with a Model-Based Design task, consider this workflow.



The workflow in this tutorial focuses on fundamental Simulink tasks as they relate to Model-Based Design.

- “System Definition and Layout” on page 1-7 — Identify modeling goals, determine components, model system layout.
- “Model and Validate a System” on page 1-13 — Model and test components, integrate components, test system.
- “Design a System in Simulink” on page 1-25 — Design and test new components.

The first two tasks in this workflow model an existing system and establish the context for designing a component. The next step in this workflow would be to implement the new component. You can use rapid prototyping and embedded code generation products, such as Simulink Real-Time™ and Embedded Coder®, to generate code and use the design with a real, physical system.

See Also

Related Examples

- “System Definition and Layout” on page 1-7
- “Model and Validate a System” on page 1-13
- “Design a System in Simulink” on page 1-25
- “Organize Large Modeling Projects Using Referenced Projects”

External Websites

- [Simulink Overview](#)
- [Model-Based Design with MATLAB and Simulink](#)

Teaching Resources

- [Engineering Problem Solving \(MathWorks Teaching Resources\)](#)

System Definition and Layout

In this section...

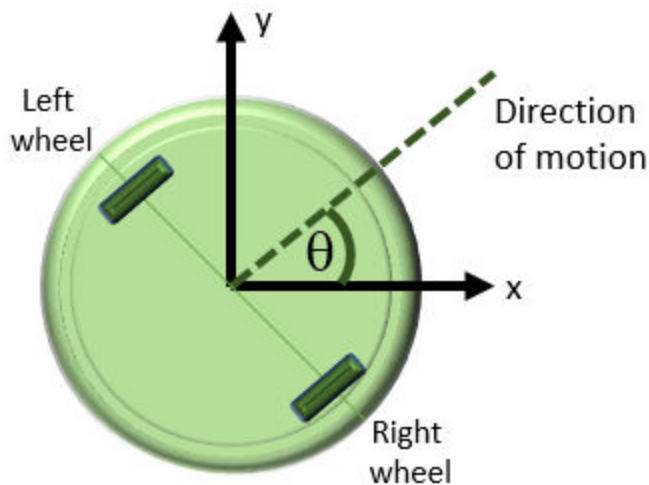
“Determine Modeling Objectives” on page 1-7

“Identify System Components and Interfaces” on page 1-8

The top-level system layout of a Simulink model is a common context that many engineering teams can use and is the basis for many tasks in the Model-Based Design paradigm: analysis, design, test, and implementation. You define a system at the top level by identifying the structure and individual components. You then organize your model in a hierarchical manner that corresponds to the components. Then you define interfaces for each component and the connections between components.

The featured model in this tutorial is a flat robot that can move or rotate with the help of two wheels, similar to a home vacuuming robot. This model assumes that the robot moves in one of two ways:

- Linear — Both wheels turn in the same direction with the same speed and the robot moves linearly.
- Rotational — The wheels turn in opposite directions with the same speed and the robot rotates in place.



Each type of motion starts from a resting state, that is, both rotational and linear speeds are zero. With these assumptions, the linear and rotational motion components can be modeled separately.

Determine Modeling Objectives

Before designing a model, consider your goals and requirements. The goals dictate both the structure and the level of detail for the model. If the goal is simply to figure out how fast the robot can go, modeling just for linear motion is sufficient. If the goal is to design a set of inputs for the device to follow a given path, then the rotational component is involved. If obstacle avoidance is a goal, then the system needs a sensor. This tutorial builds a model with the goal of designing sensor parameters so that the robot stops in time when it detects an obstacle in its path. To achieve this goal, the model must:

- Determine how quickly the robot stops when the motors stop
- Provide a series of commands for linear and rotational motion so that the robot can move in a two-dimensional space

The first modeling objective enables you to analyze the motion so you can design the sensor. The second objective enables you to test your design.

Identify System Components and Interfaces

Once you understand your modeling requirements, you can begin to identify the components of the system. Identifying individual components and their relationships within a top-level structure help build a potentially complex model systematically. You perform these steps outside Simulink before you begin building your model.

This task involves answering these questions:

- What are the structural and functional components of the system? When a layout reflects the physical and functional structure, it helps you to understand, build, communicate, and test the system. This becomes more important when parts of the system are to be implemented in different stages in the design process.
- What are the inputs and outputs for each component? Draw a picture showing the connections between components. This picture helps you to visualize signal flow within the model, identify the source and sink of each signal, and determine if all necessary components exist.
- What level of detail is necessary? Include major system parameters in your diagram. Creating a picture of the system can help you identify and model the parts that are essential to the behaviors you want to observe. Each component and parameter that contributes to the modeling goal must have a representation in the model, but there is a tradeoff between complexity and readability. Modeling can be an iterative process. You can start with a high-level model with few details and then gradually increase complexity where required.

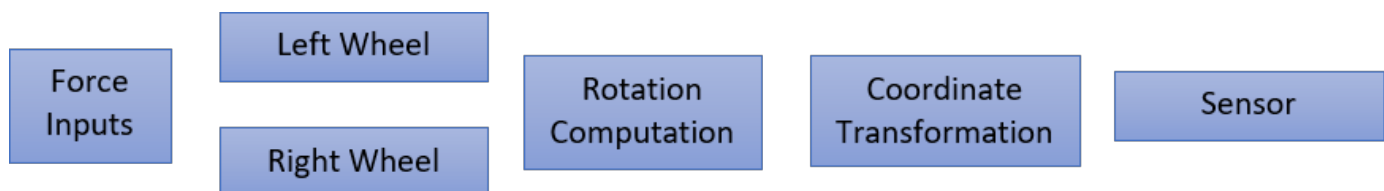
It is often beneficial to consider the following:

- What parts of the system need testing?
- What is the test data and success criteria?
- Which outputs are necessary for analysis and design tasks?

Identify Robot Motion Components

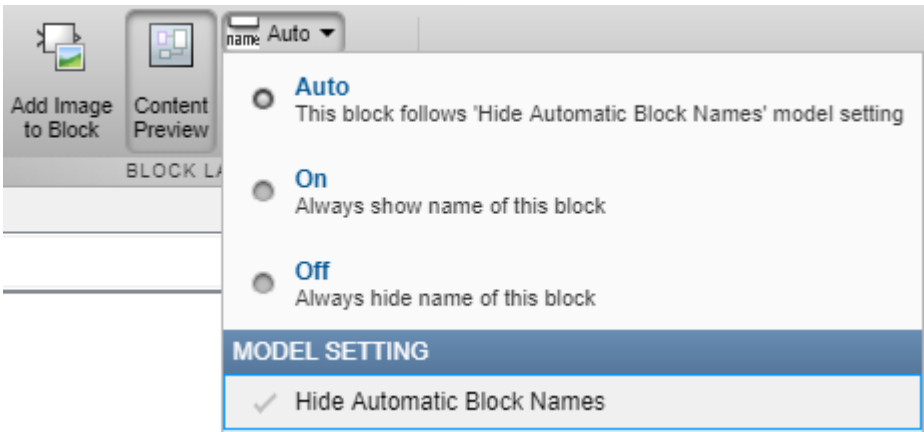
The system in this tutorial defines a robot that moves with two electric wheels in two dimensions. It includes:

- Linear motion characteristics
- Rotational motion characteristics
- Transformations to determine the location of the system in two dimensions
- A sensor to measure the distance of the robot from an obstacle

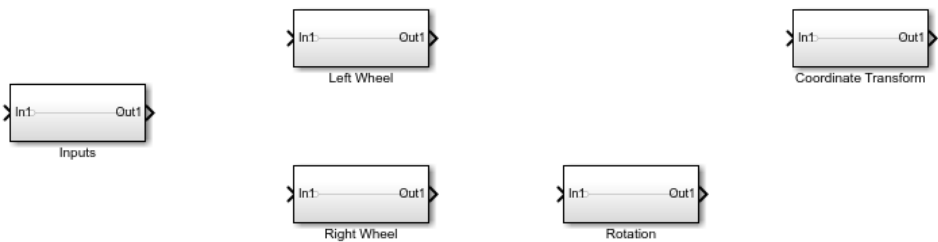


The model for this system includes two identical wheels, input forces applied to the wheels, rotational dynamics, coordinate transformation, and a sensor. The model uses a Subsystem to represent each component:

- 1 Open a new Simulink model. See “Open New Model” on page 3-3.
- 2 Open the Library Browser. See “Open Simulink Library Browser” on page 3-5.
- 3 Add Subsystem blocks. Drag five Subsystem blocks from the Ports & Subsystems library to the new model.
- 4 Click a subsystem. In the **Format** tab, click the **Auto** drop-down list. Clear the **Hide Automatic Block Names** check box.



- 5 Arrange and rename the Subsystem blocks as shown. To change block names, double-click the block name and edit the text.



Define Interfaces Between Components

Identify input and output connections between subsystems. Input and output values change dynamically during a simulation. Lines connecting blocks represent data transfer. This table shows the inputs and outputs for each component.

Block	Input	Output	Related Information
Inputs	None	Force to right wheel Force to left wheel	Not applicable
Right wheel	Force to right wheel	Right wheel velocity	Directional, negative means reverse direction

Block	Input	Output	Related Information
Left wheel	Force to left wheel	Left wheel velocity	Directional, negative means reverse direction
Rotation	Velocity difference between right and left wheels	Rotational angle	Measured counterclockwise
Coordinate Transformation	Normal speed Rotational angle	Velocity in X Velocity in Y	Not applicable
Sensor	X coordinate Y coordinate	None	No block necessary for modeling

Some block inputs do not exactly match block outputs. Therefore, in addition to the dynamics of the individual components, the model must compute the following:

- Input to the rotation computation — Subtract the velocities of the two wheels and divide by two.
- Input to the coordinate transformation — Average the velocities of the two wheels.
- Input to the sensor — Integrate the outputs of the coordinate transformation.

The wheel velocities are always equal in magnitude and the computations are accurate within that assumption.

Add the necessary components and finalize connections:

- 1 Add the necessary input and output ports to each subsystem. Double-click a Subsystem block.



Each new Subsystem block contains one Inport (In1) and one Outport (Out1) block. These blocks define the signal interface with the next higher level in a model hierarchy.

Each Inport block creates an input port on the Subsystem block, and each Outport block creates an output port. The model reflects the names of these blocks as the input/output port names. Add more blocks for additional input and output signals. On the Simulink Editor toolbar, click the

Navigate Up To Parent button 

to

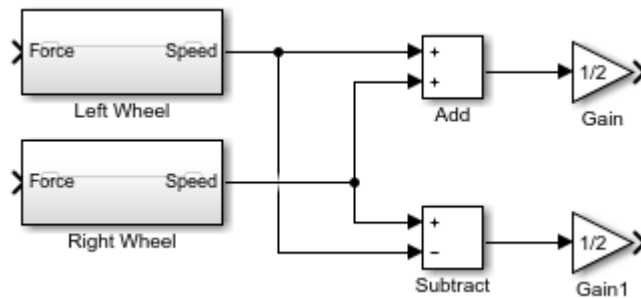
return to the top level.

For each block, add and rename Inport and Outport blocks.

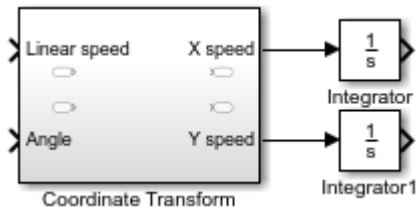


When copying an Inport block to create a new one, use the **Paste** (Ctrl+V) option.

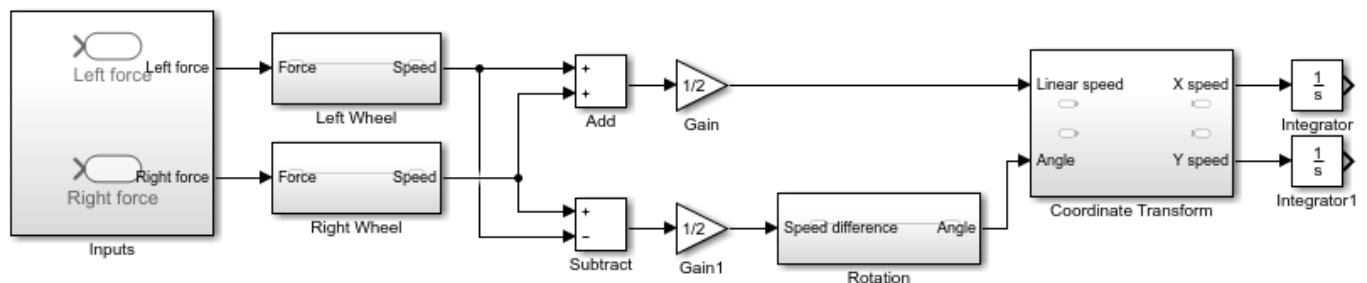
- 2 Compute required inputs to the Coordinate Transform and Rotation subsystems from the left wheel and right wheel velocities.
 - a Compute the Linear speed input to the Coordinate Transform subsystem. Add an Add block from the Math Operations library and connect the outputs of the two-wheel components. Add a Gain block and set the gain parameter to $1/2$. Connect the output of the Add block to this Gain block.
 - b Compute the Speed difference input to the Rotation subsystem. Add a Subtract block from the Math Operations library. Connect the right wheel velocity to the $+$ input and the left wheel velocity to the $-$ input. Connect the outputs of the two wheel components. Add a Gain block and set the gain parameter to $1/2$. Connect the output of the Subtract block to this Gain block.



- 3 Compute the X and Y coordinates from the X and Y velocities. Add two Integrator blocks from the Continuous library and connect the outputs of the Coordinate Transform block. Leave initial conditions of the Integrator blocks set to 0.



- 4 Complete the connections for the system.



Parameters and Data

Determine the parameters that are part of the model and their values. Use modeling goals to determine whether these values are always fixed or change from simulation to simulation. Parameters

that contribute to the modeling goal require explicit representation in the model. This table helps determine the level of detail when modeling each component.

Parameter	Block	Symbol	Value	Type
Mass	Left Wheel	m	2.5 kg	Variable
	Right Wheel			
Rolling resistance	Left Wheel	k_drag	30 Ns ² /m	Variable
	Right Wheel			
Robot radius	Rotation	r	0.15 m	Variable
Initial angle	Rotation	None	0 rad	Fixed
Initial velocities	Left Wheel	None	0 m/s	Fixed
	Right Wheel		0 m/s	
Initial (X, Y) coordinates	Integrators	None	(0, 0) m	Fixed

Simulink uses the MATLAB workspace to evaluate parameters. Set these parameters in the MATLAB command window:

```
m = 2.5;  
k_drag = 30;  
r = 0.15;
```

See Also

Related Examples

- “Model and Validate a System” on page 1-13
- “Design a System in Simulink” on page 1-25

Model and Validate a System

Model each component in the robot system to reflect its physical or functional behavior. Then, verify the model by simulating with test data. A Simulink model of a component is based on several starting points:

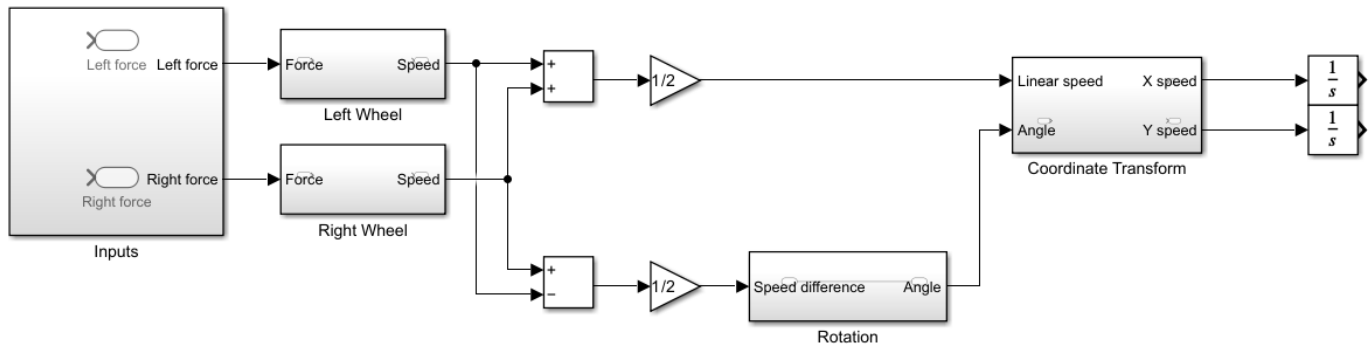
- An explicit mathematical relationship between the output and the input of a physical component — You can compute the outputs of the component from the inputs, directly or indirectly, through algebraic computations and integration of differential equations. For example, computation of the water level in a tank given the inflow rate is an explicit relationship. Each Simulink block executes based on the definition of the computations from its inputs to its outputs.
- An implicit mathematical relationship between model variables of a physical component — Because variables are interdependent, assigning an input and an output to the component is not straightforward. For example, the voltage at the positive (+) end of a motor connected in a circuit and the voltage at the negative (-) end have an implicit relationship. To model such a relationship in Simulink, you can use physical modeling tools such as Simscape™ or model these variables as part of a larger component that allows input/output definition. Sometimes, closer inspection of modeling goals and component definitions helps to define input/output relationships.
- Data obtained from an actual system — You have measured input/output data from the actual component, but a fully defined mathematical relationship does not exist. Many devices have unmodeled components that fit this description, for example, the heat dissipated by a television. You can use the System Identification Toolbox™ to define the input/output relationship for such a system.
- An explicit functional definition — You define the outputs of a functional component from the inputs through algebraic and logical computations, for example, the switching logic of a thermostat. You can model most functional relationships as Simulink blocks and subsystems.

This tutorial models physical and functional components with explicit input/output relationships. In this tutorial, you will:

- 1 Use the system equations to create a Simulink model.
- 2 Add and connect Simulink blocks in the Simulink Editor. Blocks represent coefficients and variables from the equations.
- 3 Build the model for each component separately. The most effective way to build a model of a system is to first consider components independently.
- 4 Start by building simple models using approximations of the system. Identify assumptions that can affect the accuracy of your model. Iteratively add detail until the level of complexity satisfies the modeling and accuracy requirements.
- 5 Validate components by supplying an input and observing the output.

Open System Layout

A big-picture view of the whole system layout is useful when modeling individual components. This model shows how the individual components are related to each other. Start by loading the `system_layout` model.



Model Physical Components

Describe the relationships between components, for example, data, energy, and force transfer. Use the system equations to build a graphical model of the robot system in Simulink.

Some questions to ask before you begin to model a component:

- What are the constants for each component? What values do not change unless you change them?
- What are the variables for each component? What values change over time?
- How many state variables does a component have?

Derive the equations for each component using scientific principles. Many system equations fall into three categories:

- For continuous systems, differential equations describe the rate of change for variables with the equations defined for all values of time. For example, a first-order differential equation gives the velocity of a car:

$$\frac{dv(t)}{dt} = -\frac{b}{m}v(t) + u(t),$$

where:

- $v(t)$ is the velocity of the car at time t .
- b is the damping coefficient.
- m is the mass of the car.
- $u(t)$ is the external force on the car at time t .
- For discrete systems, difference equations describe the rate of change for variables, but the equations are defined only at specific times. For example, this equation describes the control signal from a discrete proportional-derivative controller:

$$u[n] = K_p e[n] + K_d \frac{e[n] - e[n-1]}{T_s},$$

where:

- $u[n]$ is the control signal at the discrete time step n .
- $e[n]$ is the error at the discrete time step n .

- $e[n-1]$ is the error at the previous discrete time step $n-1$.
- K_p is the proportional gain.
- K_d is the derivative gain.
- T_s is the sample time.
- Equations without derivatives are algebraic equations. For example, an algebraic equation gives the total current in a parallel circuit with two components:

$$I_t = I_a + I_b,$$

where:

- I_t is the total current flowing through the circuit.
- I_a is the current flowing through branch a of the parallel circuit.
- I_b is the current flowing through branch b of the parallel circuit.

Model Linear Motion of Wheels

Two forces act on a wheel:

- Force applied by the motor — The force F acts in the direction of velocity change and is an input to the wheel subsystems.
- Drag force — The force F_{drag} acts against the direction of velocity change and is a function of velocity.

$$F_{drag} = k_{drag}V|V|$$

Acceleration is proportional to the sum of these forces:

$$\begin{aligned}(m/2)\dot{V} &= F - F_{drag} \\ (m/2)\dot{V} &= F - k_{drag}V|V| \\ \dot{V} &= \frac{F - k_{drag}V|V|}{(m/2)},\end{aligned}$$

where k_{drag} is the drag coefficient and m is the mass of the robot. Each wheel carries half of this mass.

Build the wheel model:

- 1 In the `system_layout` model, double-click the subsystem named `Right Wheel` to display the empty subsystem.
- 2 To model velocity and acceleration, add an Integrator block. Leave the initial condition set to 0. The input of this block is the acceleration $\dot{V}$ and the output is the velocity V .

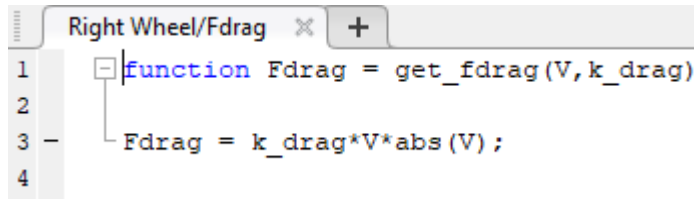


- 3 To model the drag force, add a MATLAB Function block from the User-Defined Functions library. The MATLAB Function block provides a quick way to implement mathematical expressions in your model. To edit the function, double-click the block to open the MATLAB Function Block Editor.

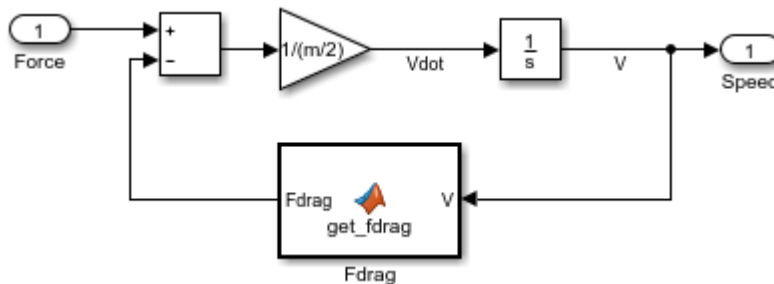
- 4 In the MATLAB Function Block Editor, enter the MATLAB code to calculate the drag force.

```
function Fdrag=get_fdrag(V,k_drag)
```

```
Fdrag=k_drag*V*abs(V);
```



- 5 Define arguments for the MATLAB Function block. In the MATLAB Function Block Editor, click Edit Data . Click k_drag , and then set **Type** to **Parameter Data**.
- 6 Subtract the drag force from the motor force using the Subtract block. Complete the force-acceleration equation using a Gain block with the **Gain** parameter set to $1/(m/2)$.
- 7 To reverse the direction of the MATLAB Function block, select the block. Then, in the Simulink Toolstrip, on the **Format** tab, click Flip left-right . Connect the blocks.



- 8 The dynamics of the two wheels are the same. Make a copy of the subsystem named Right Wheel that you just created and paste it in the subsystem named Left Wheel.

To view the parent model, click Up To Parent .

Model Rotational Motion of Wheels

When the two wheels turn in opposite directions, they move in a circle of radius r , causing rotational motion of the robot. When the wheels turn in the same direction, there is no rotation. Assuming that the wheel velocities are always equal in magnitude, you can model rotational motion as dependent on the difference of the two wheel velocities V_R and V_L :

$$\dot{\theta} = \frac{V_R - V_L}{2r}$$

Build the rotation dynamics model:

- 1 In the top level of the `system_layout` model, double-click the subsystem named Rotation to display the empty subsystem. Delete the connection between the Inport and the Outport blocks.
- 2 To model angular speed and position, add an Integrator block. Leave the initial condition set to 0. The output of this block is the rotational position θ and the input is the angular speed $\dot{\theta}$.

- 3 Compute angular speed from tangential speed. Add a Gain block and set the **Gain** parameter for the block to $1/(2*r)$.
- 4 Connect the blocks.



To view the parent model, click Up To Parent .

Model Functional Components

Describe the function from the input of a function to its output. This description can include algebraic equations and logical constructs, which you can use to build a graphical model of the system in Simulink.

Model Coordinate Transformation

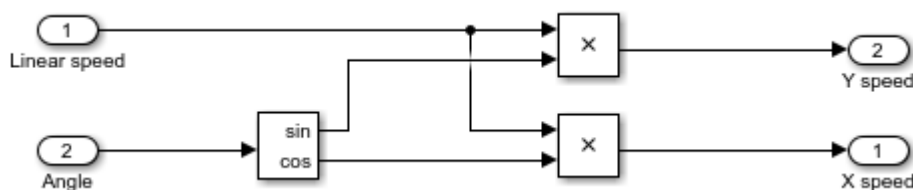
The velocity of the robot in the x and y coordinates, V_X and V_Y , is related to the linear speed V_N and the angle θ :

$$V_X = V_N \cos(\theta)$$

$$V_Y = V_N \sin(\theta)$$

Build the coordinate transformation model:

- 1 In the top level of the `system_layout` model, double-click the subsystem named `Coordinate Transform` to display the empty subsystem.
- 2 To model trigonometric functions, add a SinCos block from the Math Operations library.
- 3 To model multiplication, add two Product blocks from the Math Operations library.
- 4 Connect the blocks.



To view the parent model, click Up To Parent .

Set Model Parameters

You can determine appropriate values for parameters in your model using several sources, including:

- Written specifications such as standard property tables or manufacturer data sheets

- Direct measurements on an existing system
- Estimations using system input/output

The `system_layout` model uses these parameters.

Parameter	Symbol	Value
Mass	m	2.5 kg
Rolling resistance	k_drag	30 Ns ² /m
Robot radius	r	0.15 m

A Simulink model can access parameter values defined using variables in the MATLAB workspace. Define these variables by entering the commands in the MATLAB Command Window.

```
m = 2.5;
k_drag = 30;
r = 0.15;
```

Validate Components

Validate components by supplying an input and observing the output. Even such a simple validation can point out immediate ways to improve the model. This example validates these behaviors:

- When a force is applied continuously to a wheel, the velocity increases until it reaches a steady-state velocity.
- When the wheels turn in opposite directions, the rotation angle increases at a constant rate.

This example uses Outport blocks to log output data to the workspace and the Simulation Data Inspector. For more information about logging data and other logging techniques, see “Save Simulation Data”.

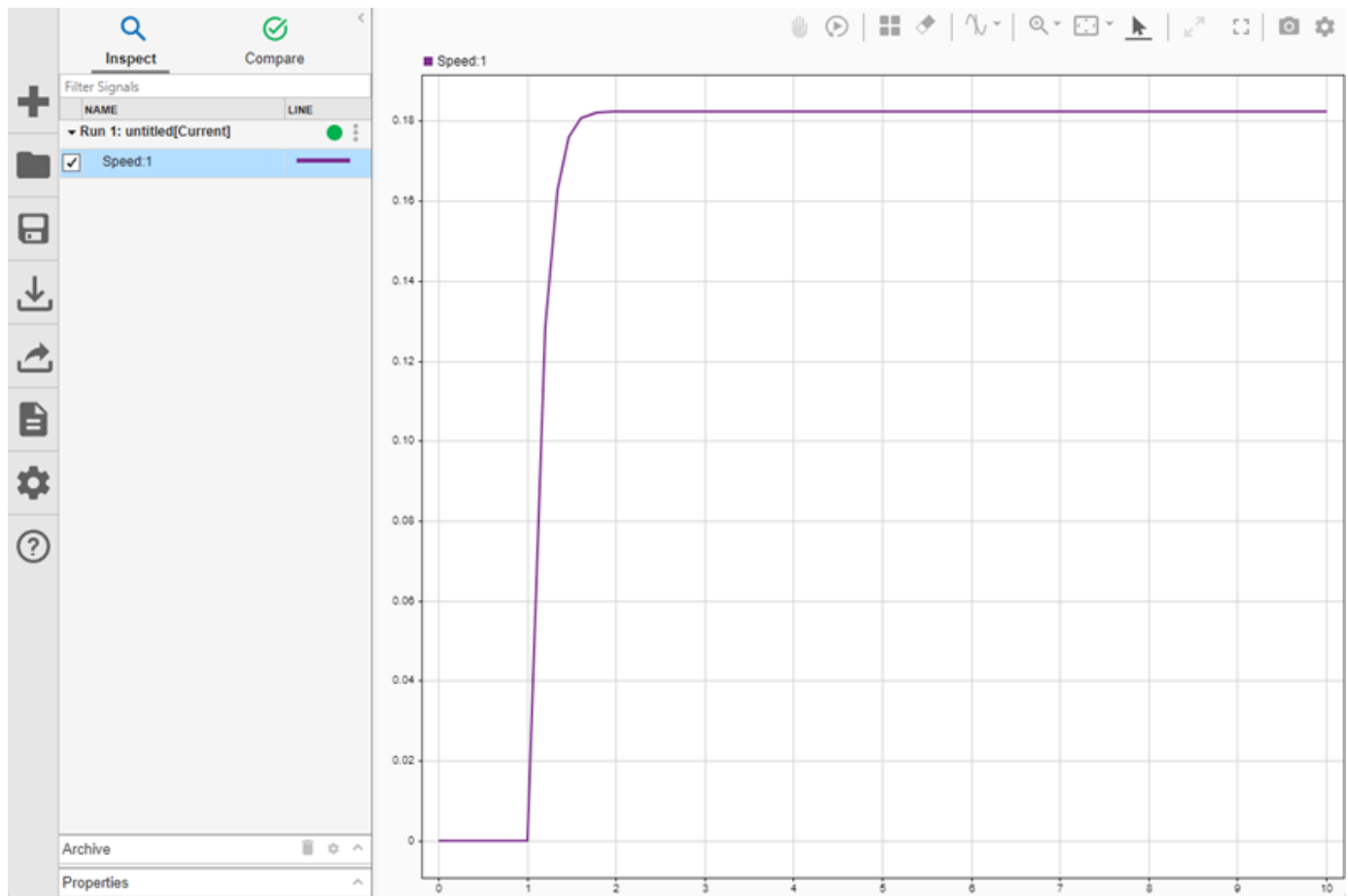
Validate Wheel Component

Create and run a test model for the wheel component:

- 1 Create a new model. In the **Simulation** tab, click **New** . Copy the Subsystem block named **Right Wheel** into the new model.
- 2 To create a test input, add a Step block from the Sources library and connect it to the input of the subsystem named **Right Wheel**. Leave the **Step time** parameter set to 1.
- 3 To log output, connect the subsystem to an Outport block.



- 4 Simulate the model. In the **Simulation** tab, click **Run** .
- 5 To view the results in the Simulation Data Inspector, click **Data Inspector** . To plot the data, select the check box next to the **Speed: 1** signal.

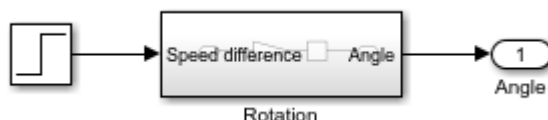


The simulation results show that the model exhibits the general expected behavior. No motion occurs until force is applied at the step time. When force is applied, the speed starts to increase and then settles at a constant when the applied force and the drag force reach an equilibrium. In addition to validation, this simulation also shows the maximum speed of the wheel for the given force.

Validate Rotation Component

Create and run a test model for the rotation model:

- 1 To create a new model, click New . Copy the subsystem block named **Rotation** into the new model.
- 2 To create a test input in the new model, add a Step block from the Sources library. Leave the **Step time** parameter set to 1. Connect the output of the Step block to the input of the subsystem named **Rotation**. This input represents the difference of the wheel velocities when the wheels rotate in opposite directions.
- 3 To log output, connect the subsystem to an Output block.

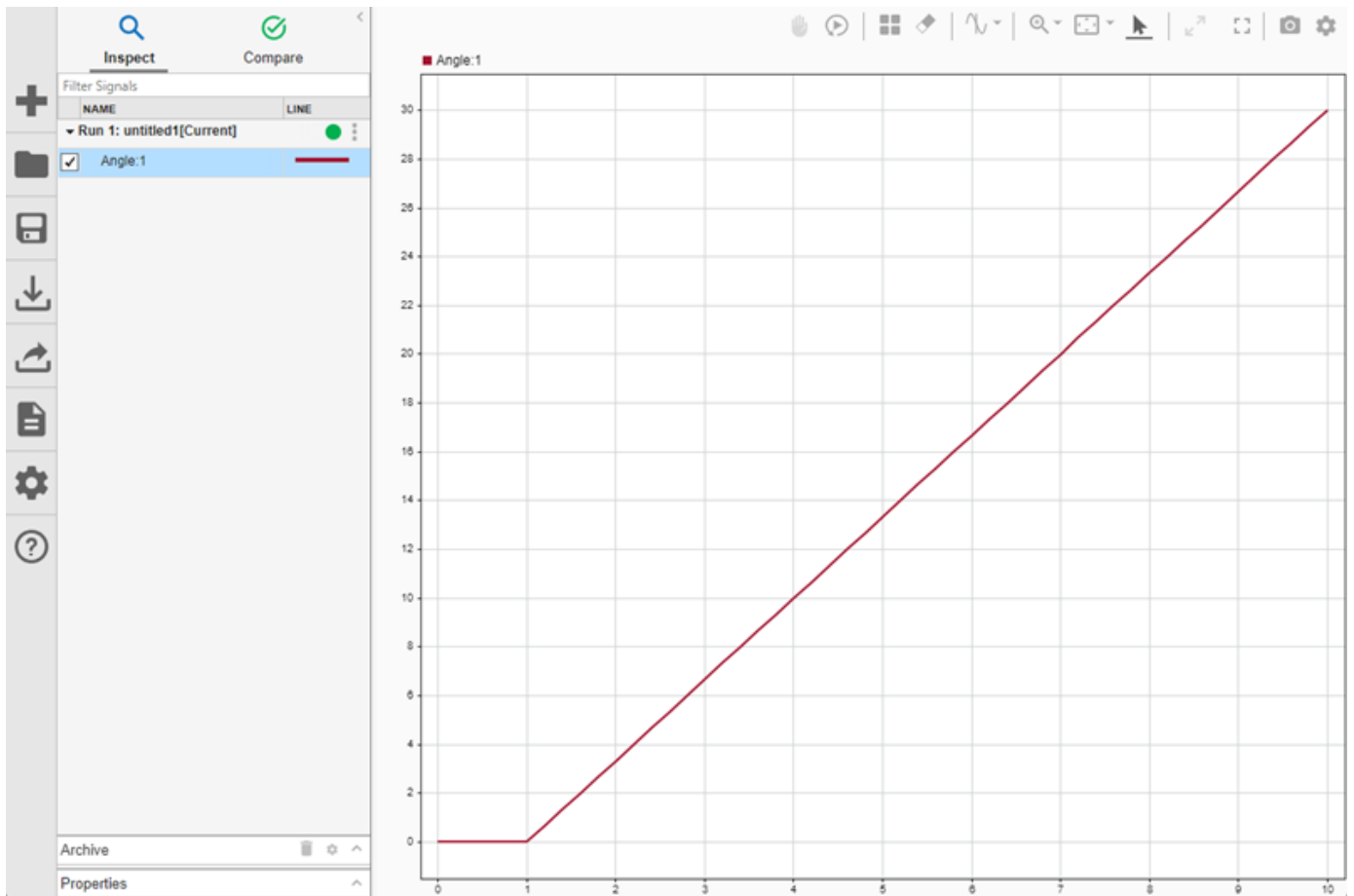


4

Simulate the model. In the **Simulation** tab, click **Run** .

5

To view the results in the Simulation Data Inspector, click **Data Inspector** . To plot the data, select the check box next to the **Angle:1** signal.



This simulation shows that the angle increases steadily when the wheels are turning with the same speed in opposite directions. You can improve the model to make the angle output easier to interpret. For example, you can:

- Convert the output signal units from radians to degrees by adding a Gain block with a gain of $180/\pi$.
- Display the output value in cycles of 360 degrees by adding a Math Function block with function `mod`.

MATLAB trigonometric functions take inputs in radians.

Validate Model

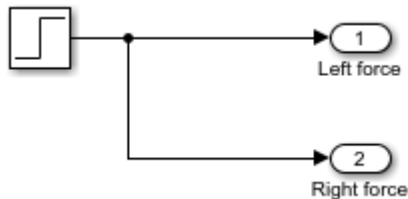
After you validate individual components, you can perform a similar validation on the complete model. This example validates these behaviors:

- When the same force is applied to both wheels in the same direction, the robot moves in a line.
- When the same force is applied to both wheels in opposite directions, the robot rotates in place.

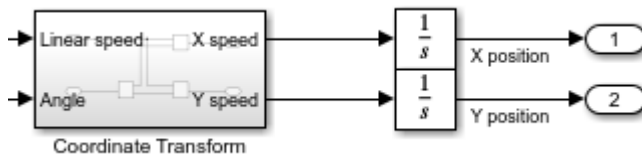
Test Wheels Spinning in Same Direction

When the wheels are both spinning in the same direction, the robot should move in a line.

- 1 In the `system_layout` model, double-click the subsystem named `Inputs` to display the empty subsystem.
- 2 Create a test input by adding a Step block. Leave the **Step time** parameter set to 1. Connect the Step block output to both Output blocks.

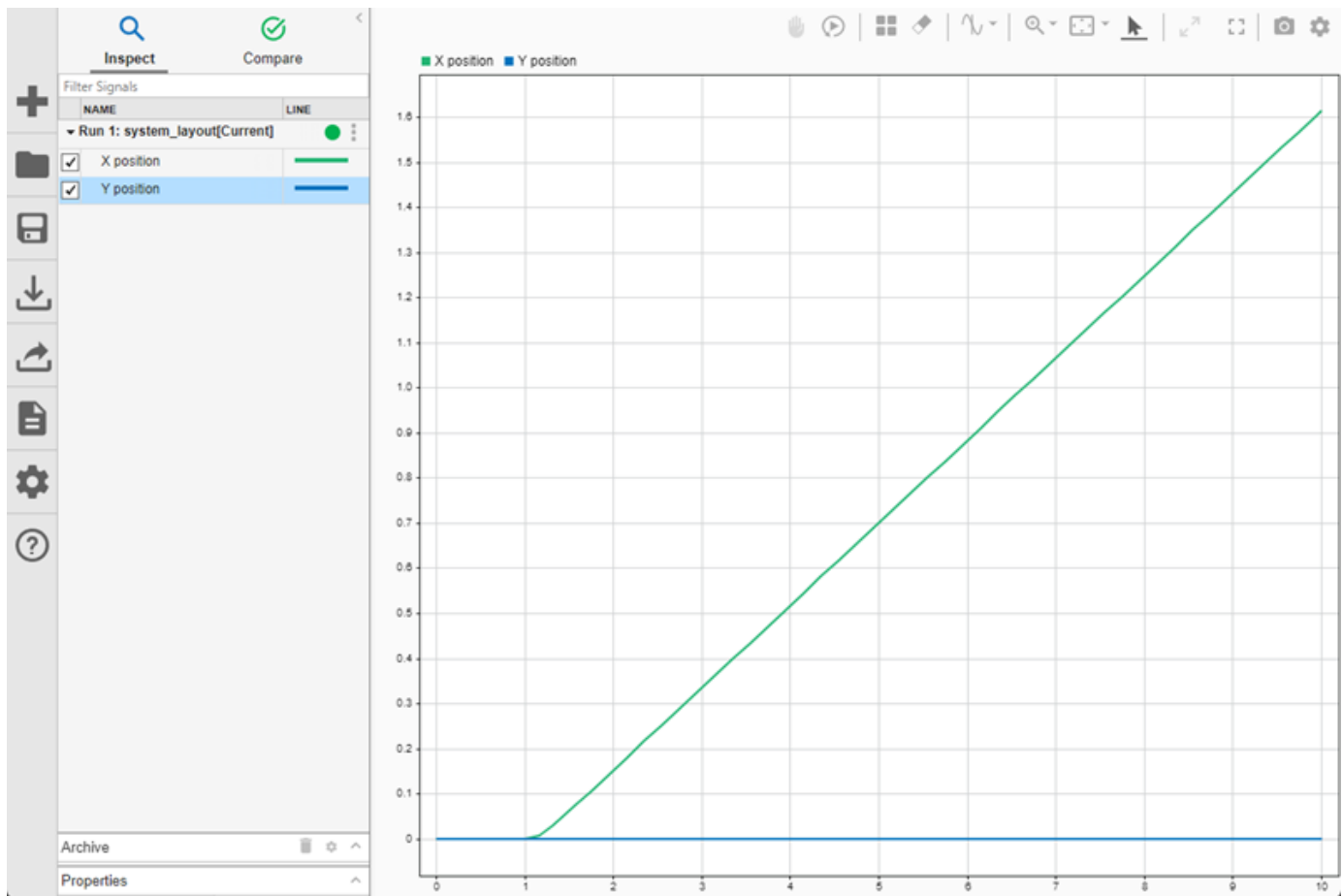


- 3 At the top level of the model, connect both Integrator blocks to Output blocks to log the data. Label the signals `X position` and `Y position`.



- 4 Simulate the model.
- 5 In the Simulation Data Inspector, select the `X position` and `Y position` signals to view the results.

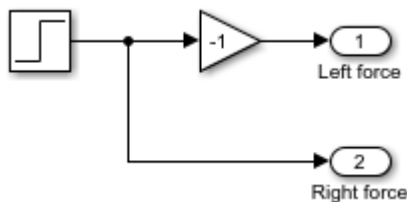
Since the angle is zero and does not change, the vehicle moves only in the x direction, as expected.



Test Wheels Spinning in Opposite Directions

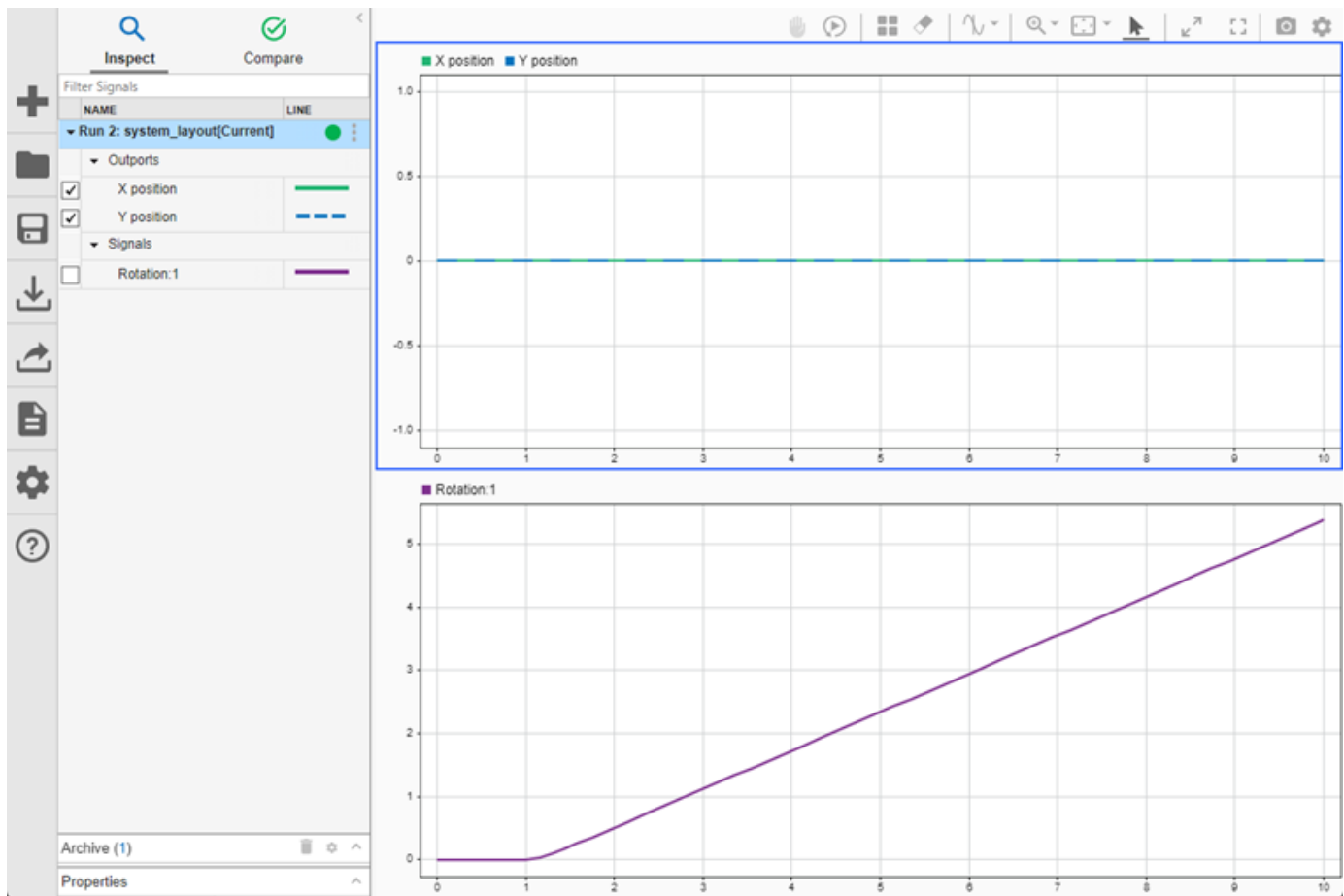
When the wheels spin in opposite directions, the robot should rotate in place.

- 1 Double-click the subsystem named **Inputs**. To reverse the direction for the left wheel, add a Gain block between the source and the second output and set the **Gain** parameter to -1.



- 2 Mark the angle output for signal logging. Right-click the output signal from the Subsystem block named **Rotation**, and then click the Log Signals button .
- 3 Simulate the model.
- 4 View the signals in the Simulation Data Inspector using two vertically aligned subplots. Click Visualizations and Layouts . Then, under **Basic Layouts**, choose the 2× 1 layout. In the upper subplot, plot the X position and Y position signals. In the lower subplot, plot the Rotation:1 signal.

The upper subplot shows that no motion occurs in the X-Y plane. The lower subplot shows steady angular motion.



You can use this final model to explore different scenarios by changing the input:

- Observe what happens when you set the initial angle to a value other than zero.
- Note how long it takes for the motion to stop when the force drops to zero.
- Increase the weight of the robot. Observe what happens when the robot is heavier.
- See how the robot moves on a smoother surface with a smaller drag coefficient.

See Also

Blocks

Pulse Generator | Gain | Integrator | Sum | Constant | Zero-Order Hold | Subsystem | MATLAB Function

Tools

Simulation Data Inspector | MATLAB Function Block Editor

See Also

Related Examples

- Outport
- “System Definition and Layout” on page 1-7
- “Design a System in Simulink” on page 1-25
- “Save Simulation Data”
- “Create Plots Using Simulation Data Inspector”

Design a System in Simulink

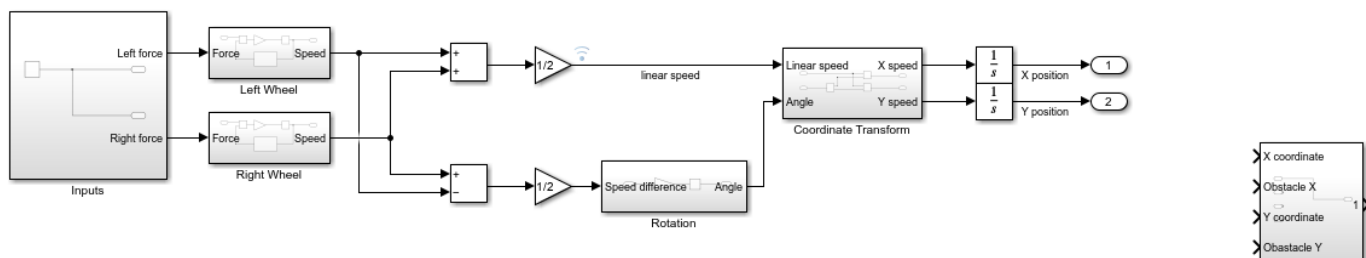
In this section...

“Open System Model” on page 1-25
 “Identify Designed Components and Design Goals” on page 1-25
 “Analyze System Behavior Using Simulation” on page 1-26
 “Design Components and Verify Design” on page 1-28

Model-Based Design centers on models of physical components and systems as a basis for design, testing, and implementation activities. This tutorial shows you how to add a designed component to an existing system model by adding an alert system to a model of a flat robot.

Open System Model

This model is a flat robot that can move or rotate with the help of two wheels, similar to a home vacuuming robot.



Identify Designed Components and Design Goals

Specifying the design objective is a critical first step to the design task. Even a simple system can have multiple and even competing design goals. Consider these goals for the example model:

- Design a controller that varies the force input so that the wheels turn at a desired speed.
- Design inputs that make the device move in a predetermined path.
- Design a sensor and controller so that the device follows a line.
- Design a planning algorithm so that the device reaches a certain point using the shortest path possible while avoiding obstacles.
- Design a sensor and algorithm so that the device moves over a certain area while avoiding obstacles.

This tutorial shows you how to design an alert system. Use the existing model to determine the parameters needed for a sensor that measures the distance from an obstacle. This example assumes a perfect sensor that measures the distance accurately. An alert system samples these measurements at fixed intervals to keep the output within 0.05 m of the measurement, triggering an alert in time for the robot to stop before a hitting the obstacle.

Analyze System Behavior Using Simulation

The design of the new component requires analyzing the linear motion of the robot to determine:

- How far the robot can travel at the top speed when power to the wheels is cut
- The top speed of the robot

Simulate the model with a force input signal that starts motion, waits until the robot reaches a steady velocity, and then sets the force to zero:

- 1 In the model, double-click the subsystem named **Inputs**.
- 2 Delete the existing step input and add a Pulse Generator block.
- 3 Set these parameters for the Pulse Generator block:

- **Amplitude:** 1
- **Period:** 20
- **Pulse Width:** 15

These parameter values help the robot to reach top speed. You can change parameters to see how different values affect the movement of the robot.

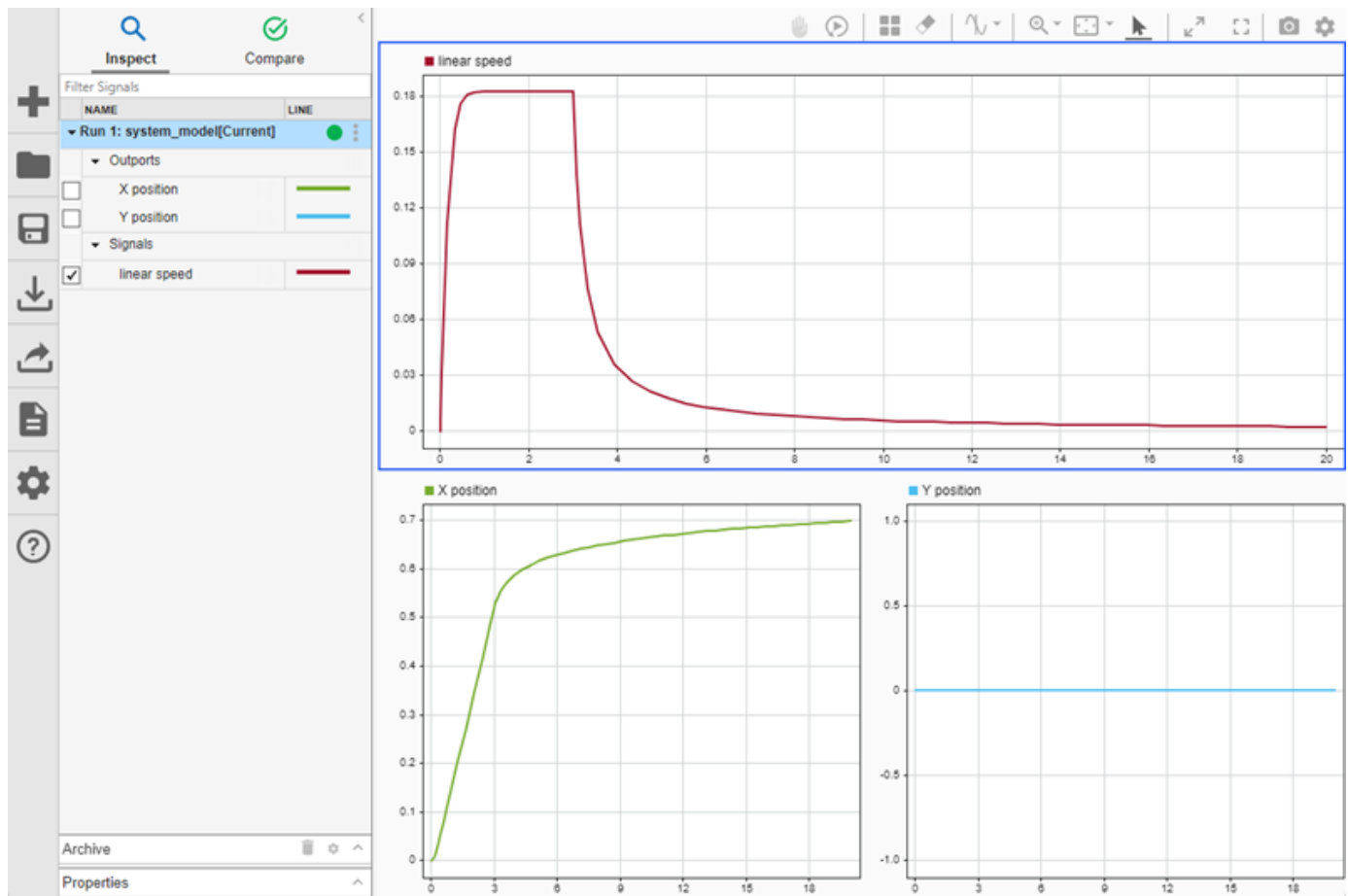
- 4 Simulate the model for 20 seconds.
- 5 Analyze the simulation results. To open the Simulation Data Inspector, click **Data Inspector**. This model logs data for the `linear speed`, `X position`, and `Y position` signals.

Change the subplot layout to view all three signals. Click the Visualizations and Layouts button



. Then, under **Basic layouts**, choose the split bottom layout

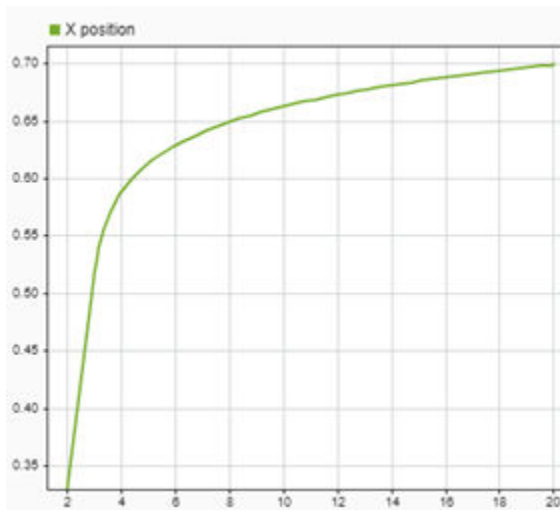
- In the top subplot, plot the `linear speed` signal.
- In the bottom-left subplot, plot the `X position` signal.
- In the bottom-right subplot, plot the `Y position` signal.



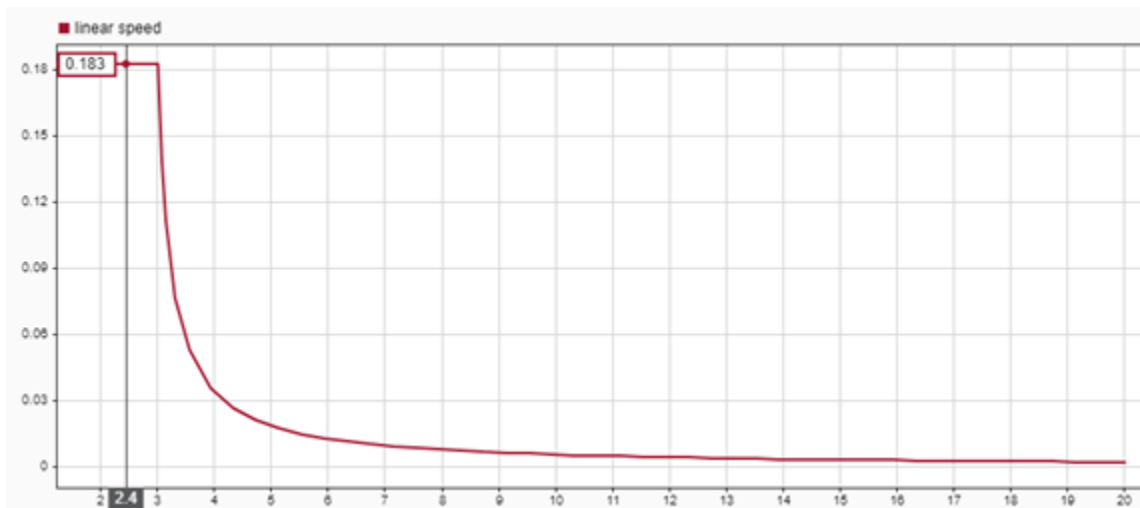
The upper subplot shows that the speed of the robot rapidly decreases after the pulse that represents the input force drops to 0 at a simulation time of 3 seconds. The speed asymptotically approaches 0 but never reaches 0. Accurate modeling for low-speed dynamics without external forces requires a more complex representation of the system. However, this approximate representation of the system is sufficient for the purpose of the example.

The lower subplots show the x and y position of the robot over the course of the simulation. The robot does not move in the y direction. In the x direction, at the start of the simulation, the position changes rapidly. At about a simulation time of 3 seconds, the position changes more slowly as the speed of the robot decreases.

You can zoom and pan to find the final position of the robot. Click the Zoom in button , and then click and drag to zoom in on the bottom-left subplot that shows the robot position. At time 3 seconds, the position of the robot is approximately 0.55 m. At the end of the simulation, the position of the robot is approximately 0.7 m. Because the speed of the robot is close to zero at the end of the simulation, the results show that the robot moves less than 0.16 m after the external force drops to zero.



You can use cursors to find the top speed. Click the Show/hide data cursors button . Position the cursor in the region where the velocity curve is flat. The data cursor label shows that the top speed of the robot is 0.183 m/s. To calculate how long the robot takes to travel 0.05 m, divide 0.05 m by 0.183 m/s to get the result of 0.27 sec.



Design Components and Verify Design

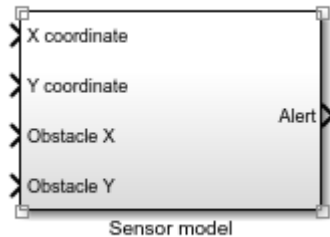
Sensor design consists of these components:

- Measurement of the distance between the robot and the obstacle — This example assumes that the measurement is perfect.
- The time interval between each distance measurement the alert system makes — To keep the measurement error below 0.05 m, the sampling interval must be less than 0.27 sec. Use 0.25 sec.
- The distance at which the sensor produces an alert — Analysis shows that slow down must start by the time the robot is approximately 0.16 m from the obstacle. The actual alert distance must also take the error from discrete measurements, 0.05 m, into account.

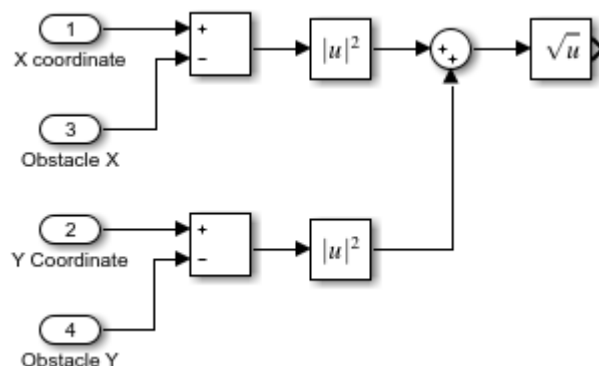
Add Designed Component

Build the sensor:

- 1 Create a subsystem with four input ports and one output port. The subsystem receives inputs for the x- and y-coordinates of the robot and for the x- and y-coordinates of the obstacle. The alert signal produced by the sensor connects to the output port.



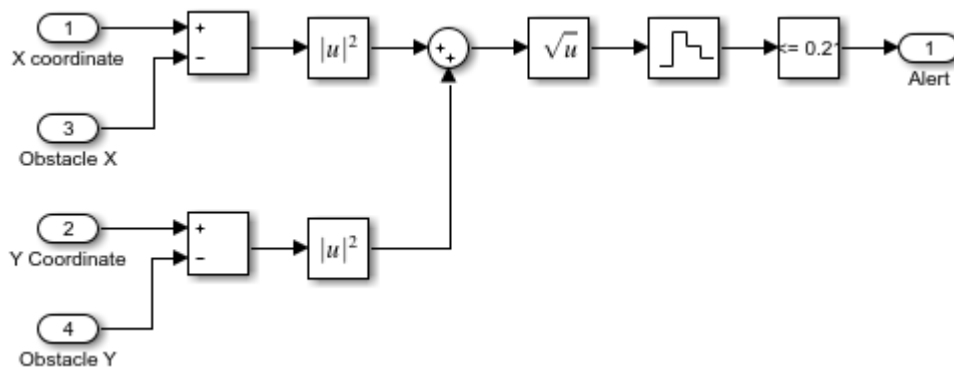
- 2 Construct the distance measurement subsystem. In the subsystem named `Sensor model`, use the Subtract block, the Math Function block with the function `magnitude^2`, the Sum block, and the Sqrt block to implement the distance calculation. Notice that within the subsystem, the arrangement of the input ports does not need to match the arrangement of the ports on the Subsystem block interface.



- 3 To model the sampling, add a Zero-Order Hold block to the subsystem from the Discrete library and set the **Sample time** parameter for the block to 0.25.
- 4 Connect the result of the distance calculation to the input of the Zero-Order Hold block.
- 5 To model the alert logic, add a Compare to Constant block from the Logic and Bit Operations library and set these block parameters:
 - **Operator:** `<=`
 - **Constant Value:** 0.21
 - **Output data type:** boolean

With these parameter values, the block output value is 1 when the input value, which indicates the distance between the robot and the obstacle, is less than or equal to 0.21.

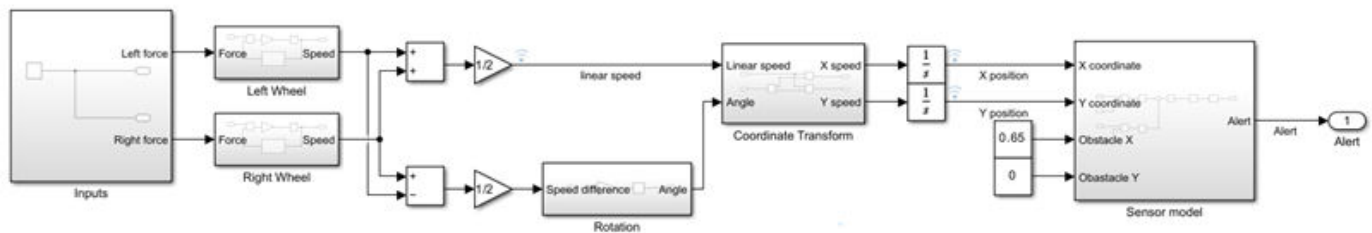
- 6 Connect the output of the Zero-Order Hold block to the input of the Compare to Constant block.
- 7 Finally, connect the output of the Compare to Constant block to the Outport block named Alert.



Verify Design

Test the design with an obstacle location of $X = 0.65$, $Y = 0$ using Constant blocks as inputs to the `Sensor model` subsystem. This test verifies design functionality in the X direction. You can create similar tests for different paths. This model only generates an alert and does not control the robot.

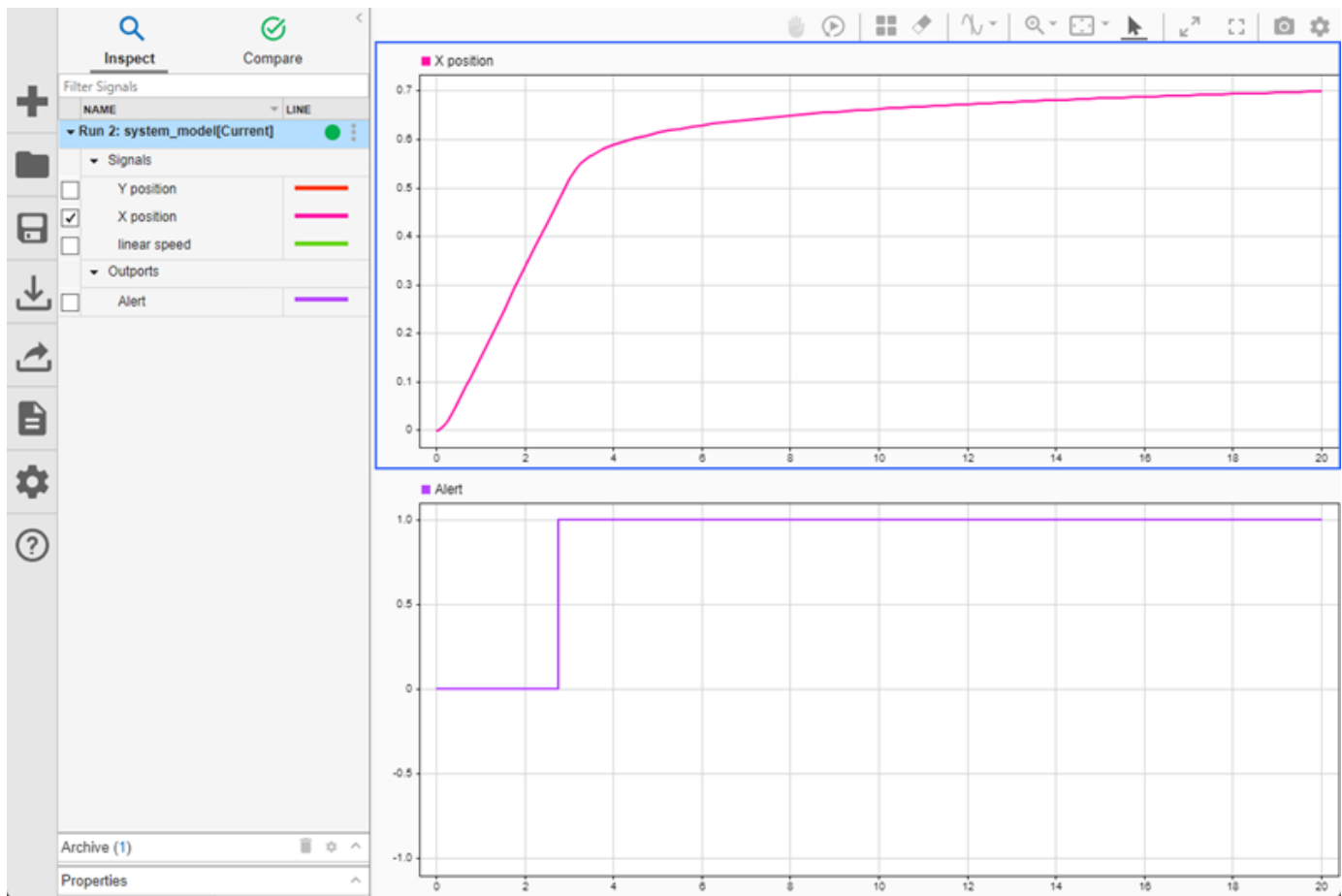
- 1 Set the obstacle location. Add two Constant blocks from the Sources library set the constant values to 0.65 and 0 . Connect the position outputs of the robot to the inputs of the sensor.
- 2 Connect an Output block to the `Sensor model` subsystem to log the data.
- 3 Mark the X position and Y position signals for signal logging. Select both signals. On the **Simulation** tab, click **Log Signals**.



- 4 Simulate the model.

In the Simulation Data Inspector, click the Visualizations and Layouts button  to change the plot layout to a 2×1 layout. In the upper subplot, plot the X position signal. In the lower subplot, plot the Alert signal.

The plot of the robot position looks the same as in the previous run. The plot of the alert signal shows that the alert signal value becomes 1 when the robot comes within 0.21 m of the obstacle location, satisfying the design requirement for this component.



For real-world systems with complex components and formal requirements, the Simulink product family includes additional tools to refine and automate the design process. Requirements Toolbox™ provide tools to formally define requirements and link them to model components. Simulink Control Design™ can facilitate the design if you want to build a controller for this robot. Simulink Verification and Validation™ products establish a formal framework for testing components and systems.

See Also

Blocks

Pulse Generator | Gain | Integrator | Sum | Constant | Zero-Order Hold | Subsystem

Tools

Simulation Data Inspector

See Also

Related Examples

- “Model-Based Design with Simulink” on page 1-3
- “System Definition and Layout” on page 1-7

- “Model and Validate a System” on page 1-13
- “Save Simulation Data”
- “Create Plots Using Simulation Data Inspector”

Modeling in Simulink

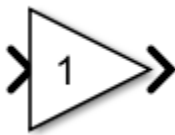
Simulink Block Diagrams

Simulink is a graphical modeling and simulation environment for dynamic systems. You can create block diagrams, where blocks represent parts of a system. A block can represent a physical component, a small system, or a function. An input/output relationship fully characterizes a block. Consider these examples:

- A faucet fills a bucket — Water goes into the bucket at a certain flow rate, and the bucket gets heavier. A block can represent the bucket, with flow rate as the input and its weight as the output.
- You use a megaphone to make your voice heard — The sound produced at one end of the megaphone is amplified at the other end. The megaphone is the block, the input is the sound wave at its source, and the output is the sound wave as you hear it.
- You push a cart and it moves — The cart is the block, the force you apply is the input, and the cart's position is the output.

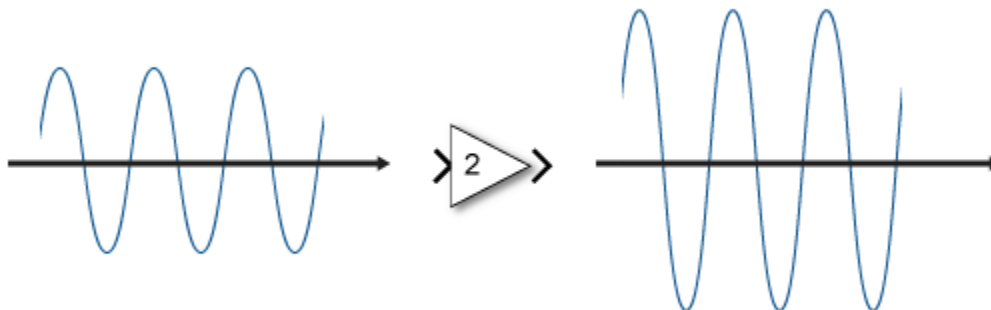
The definition of a block is only complete with its inputs and outputs defined; this task relates to the goal of the model. For example, the cart velocity may be a natural choice as an output if the modeling goal does not involve its location.

Simulink provides block libraries that are collections of blocks grouped by functionality. For example, to model a megaphone that multiplies its input by a constant, you use a Gain block from the Math Operations library.



A sound wave goes into the megaphone as its input, and a louder version of the same wave comes out as its output.

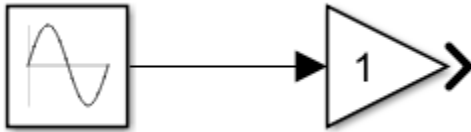
The > signs denote the inputs and outputs of a block, which can be connected to other blocks.



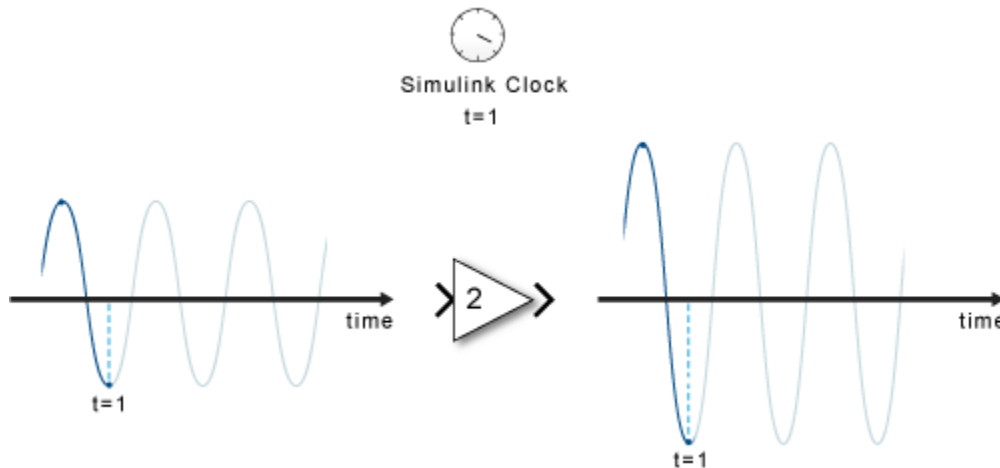
You can connect blocks to other blocks to form systems and represent more complex functionality. For example, an audio player turns a digital file into sound. A digital representation is read from storage,

is interpreted mathematically, and then turned into physical sound. The software that processes the digital file to compute the sound waveform can be one block; the speaker that takes the waveform and turns it into sound can be another block. A component that generates the input is another block.

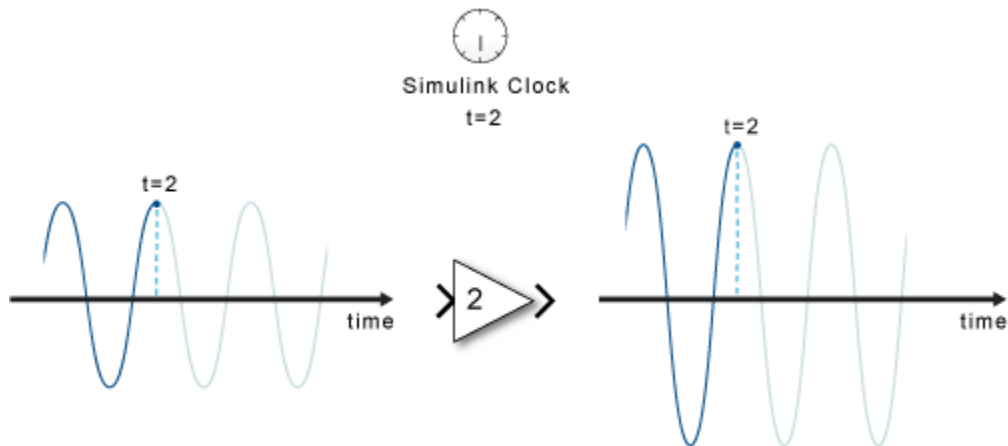
To model the sine wave input to the megaphone in Simulink, include a Sine Wave source.



The primary function of Simulink is to simulate behavior of system components over time. In its simplest form, this task involves keeping a clock, determining the order in which the blocks are to be simulated, and propagating the outputs computed in the block diagram to the next block. Consider the megaphone. At each time step, Simulink must compute the value of the sine wave, propagate it to the megaphone, and then compute the value of its output.



At each time step, each block computes its outputs from its inputs. Once all of the signals in a diagram are computed at a given time step, Simulink determines the next time step (based on the model configuration and numerical solver algorithms) and advances the simulation clock. Then each block computes their output for this new time step.



In simulation, time progresses differently from a real clock. Each time step takes as much time as it takes to finish the computations for that time step, whether that time step represents a fraction of a second or a few years.

Often, the effect of a component's input on its output is not instantaneous. For example, turning on a heater does not result in an instant change in temperature. Rather, this action provides input to a differential equation. The history of the temperature (a *state*) is also a factor. When simulation requires solving a differential or difference equation, Simulink employs memory and numerical solvers to compute the state values for the time step.

Simulink handles data in three categories:

- Signals — Block inputs and outputs, computed during simulation
- States — Internal values, representing the dynamics of the block, computed during simulation
- Parameters — Values that affect the behavior of a block, controlled by the user

At each time step, Simulink computes new values for signals and states. By contrast, you specify parameters when you build the model and can occasionally change them while simulation is running.

See Also

Related Examples

- “Create a Simple Model” on page 3-2
- “Model-Based Design with Simulink” on page 1-3

Simple Simulink Model

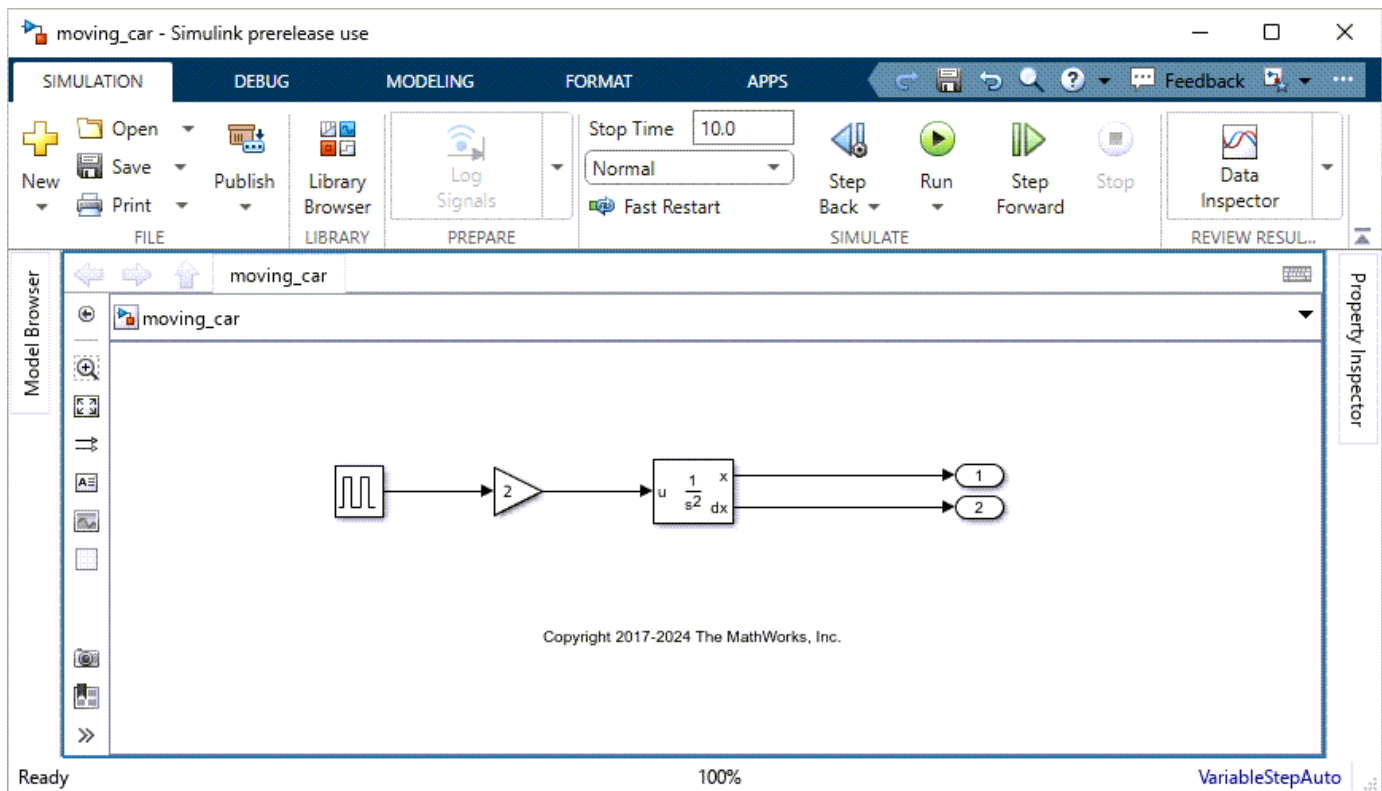
Create a Simple Model

In this section...
“Open New Model” on page 3-3 “Open Simulink Library Browser” on page 3-5 “Add Blocks to Model” on page 3-7 “Connect Blocks” on page 3-9 “Edit Block Parameter Values” on page 3-11 “Run Simulation” on page 3-12 “View Simulation Data” on page 3-12 “Refine Model” on page 3-13 “Provide Feedback on the Software” on page 3-19

You can use Simulink to model a system and then simulate the dynamic behavior of that system. In this example, you create a simple model, but you can use the same basic techniques to create complex models. The example simulates the simplified motion of a car. A car is typically in motion while the gas pedal is pressed. After the pedal is released, the car slows to a stop and idles.

A Simulink block is a model element that defines a mathematical relationship between its input and output. To create this simple model, you need four Simulink blocks.

Block Name	Block Purpose	Model Purpose
Pulse Generator	Generate an input signal for the model.	Represent the accelerator pedal.
Gain	Multiply the input signal by a constant value.	Calculate how pressing the accelerator affects the car acceleration.
Second-Order Integrator	Integrate the input signal twice.	Obtain the position from acceleration.
Outputport	Designate a signal as an output from the model.	Designate the position as an output from the model.



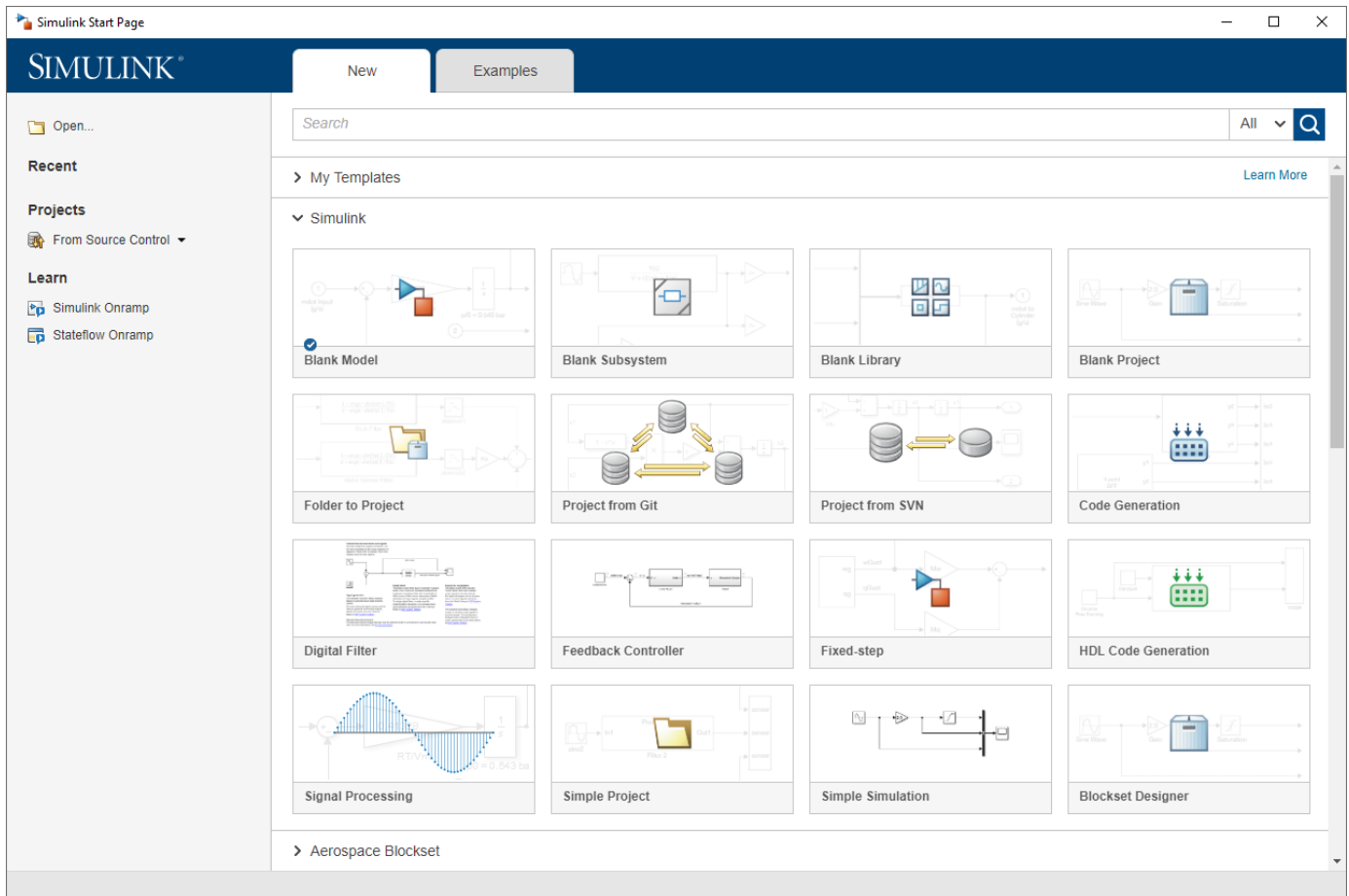
Simulating this model integrates a brief pulse twice to get a ramp. The input pulse represents a press of the gas pedal: 1 when the pedal is pressed and 0 when it is not. The output ramp is the increasing distance from the starting point.

Open New Model

Use the Simulink Editor to build your models.

- 1 Start MATLAB. From the MATLAB Toolstrip, click the **Simulink** button

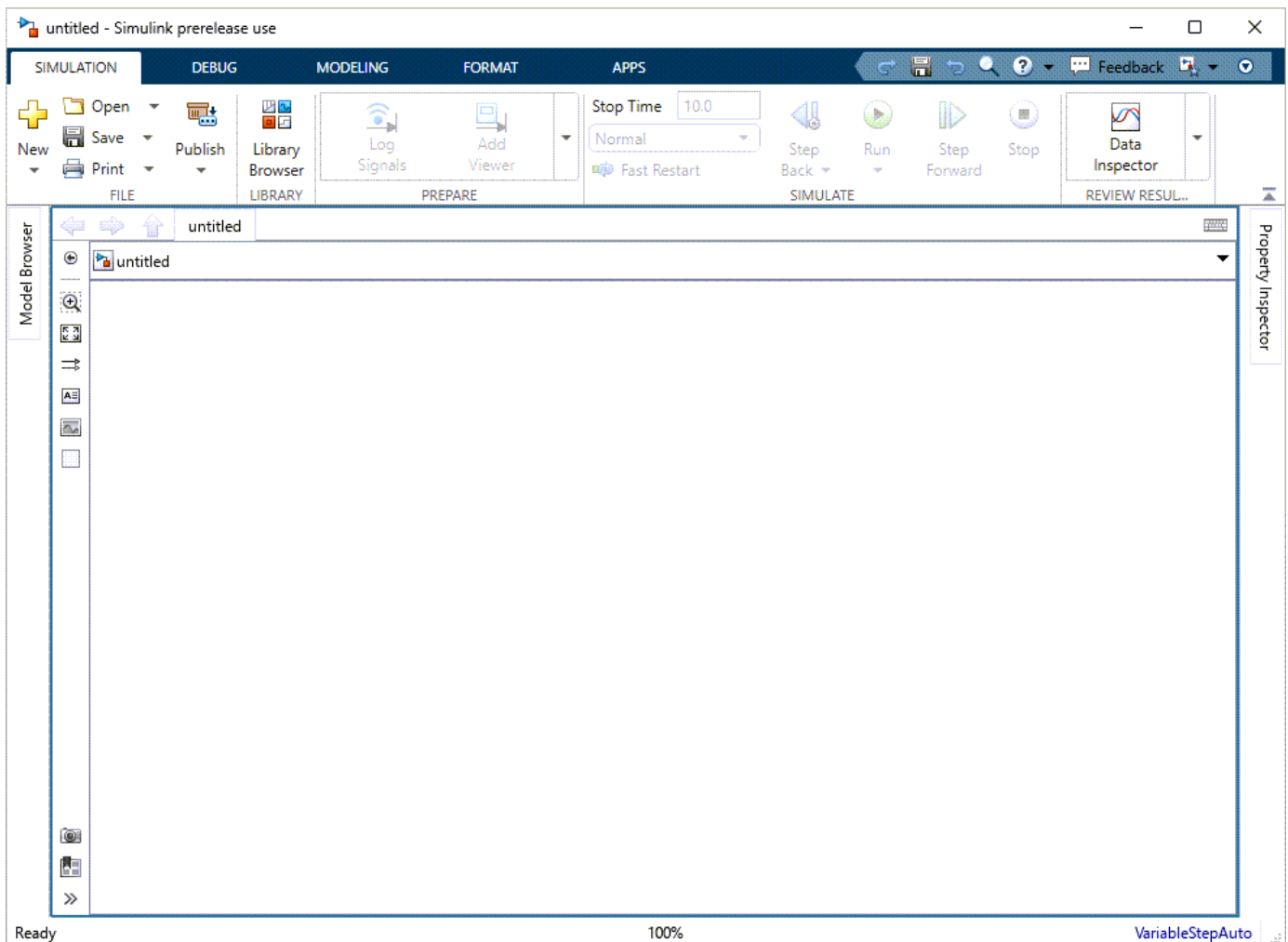




- 2 Select the **Blank Model** template.

The Simulink Editor opens.

To avoid shadowing, or having more than one model with the same name open at once, the Simulink Editor checks loaded models and files on the path and creates a model with the next available name: untitled, untitled1, untitled2, and so on.



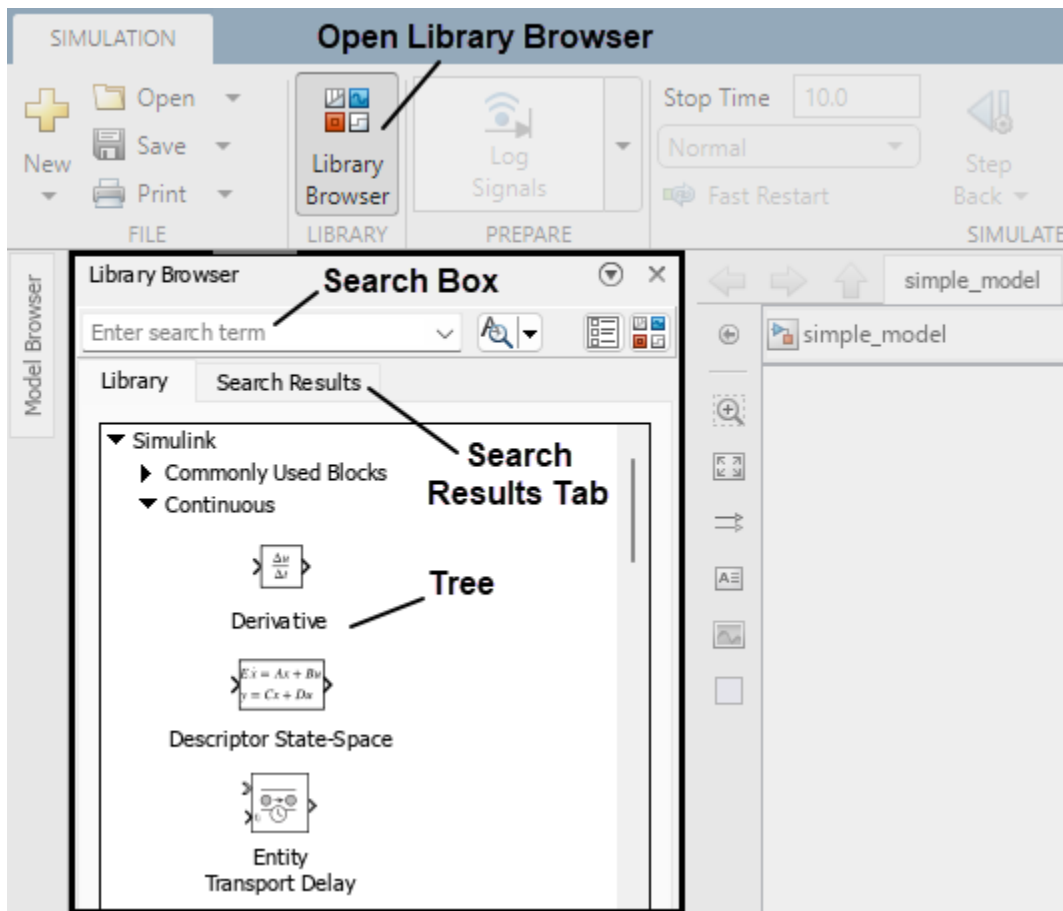
- 3 On the **Simulation** tab, select **Save > Save as**. In the **File name** text box, enter a name for your model, for example, `simple_model`. Click **Save**. The model is saved with the file extension `.slx`.

Open Simulink Library Browser

Simulink provides a set of block libraries that are organized by functionality in the Library Browser. These libraries are common to most workflows:

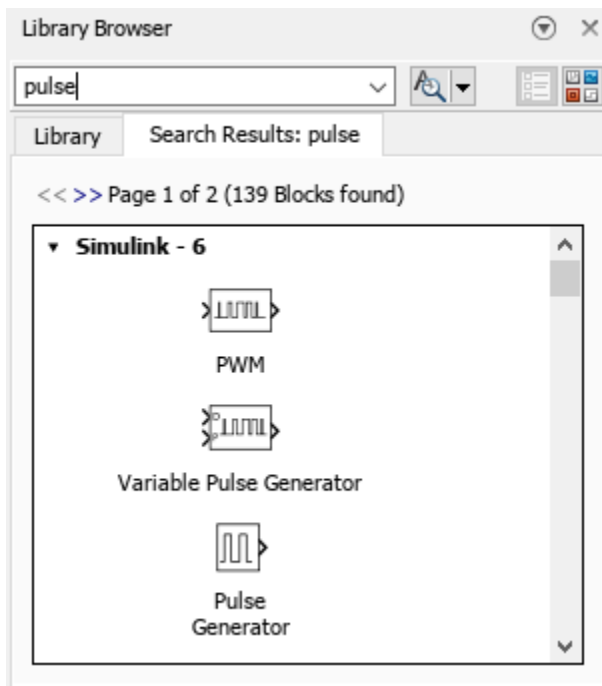
- Continuous — Blocks for systems with continuous states
- Discrete — Blocks for systems with discrete states
- Math Operations — Blocks that implement algebraic and logical equations
- Sinks — Blocks that store and show the signals that connect to them
- Sources — Blocks that generate the signal values that drive the model

To open the Library Browser, in the Simulink Toolstrip, on the **Simulation** tab, click **Library Browser**.



To browse through the block libraries, in the library tree, expand a library and its sublibraries.

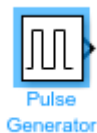
To search all of the available block libraries, enter a search term. For example, find the Pulse Generator block. In the search box, enter **pulse**, then press **Enter**. The software searches the libraries for blocks with **pulse** in their name or description and then displays the blocks on the **Search Results** tab of the Library Browser. You can return to browsing the library tree by clicking the **Library** tab.



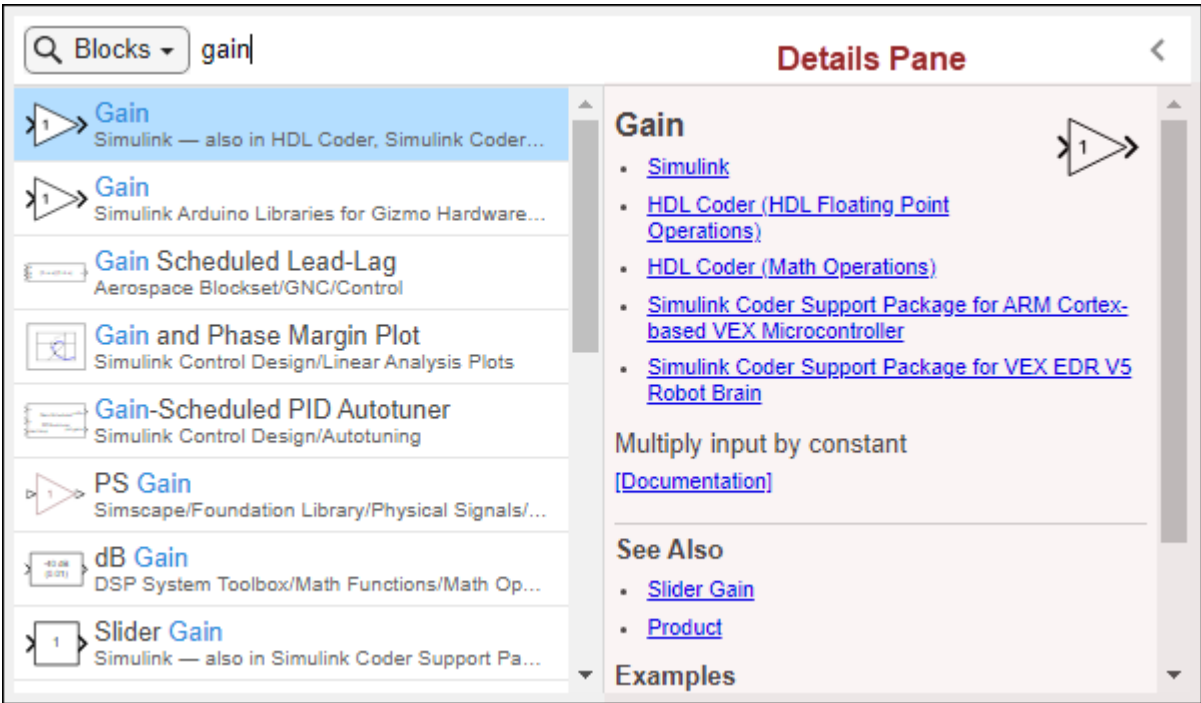
Add Blocks to Model

To start building the model, add blocks to the model canvas. You can add blocks using the Library Browser or the quick insert menu.

- 1 Add a Pulse Generator block. In the Library Browser tree, expand the Simulink library. Expand the Sources sublibrary. Drag the Pulse Generator block to the model canvas.



- 2 Add a Gain block. Double-click the model canvas. In the quick insert menu that appears, enter gain. A list of blocks appears.



Multiple different blocks can have the same name, provided they are stored in different library files. The libraries to which a block belongs are listed under the block name. Check that the Gain block from the Simulink library is selected. If not, select the block using your arrow keys or by clicking the block name.

To learn about the selected block, read the description in the details pane to the right of the search results. To see the full documentation for the block, click **Documentation**. For examples involving the block, click the links under **Examples**. To hide or unhide the details pane, click the arrow  in the upper right.

Add the selected block to the model by pressing **Enter** or double-clicking the selection.



- 3 Add these blocks to the model using the Library Browser or the quick insert menu.

Block	Library
Out1	Simulink library, Sinks sublibrary
Second-Order Integrator	Simulink library, Continuous sublibrary

- 4 Add a second Out1 block by copying the existing one and pasting it at another point using the keyboard shortcuts **Ctrl+C** and **Ctrl+V** (on macOS, use **command+C** and **command+V**). Your model now has the blocks you need.

Connect Blocks

Connect the:

- Pulse Generator block to the Gain block.
- Gain block to Second-Order Integrator block.
- Second-Order Integrator block to the two Out1 blocks.

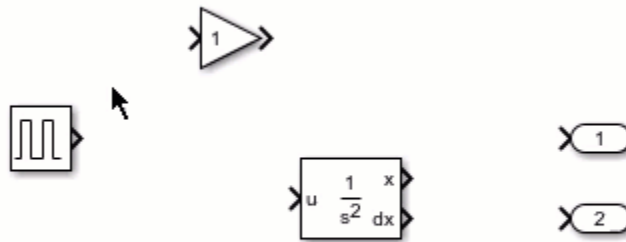
For example, to connect the Pulse Generator block to the Gain block:

- 1 Click the output port on the right side of the Pulse Generator block.

The output port and all input ports suitable for a connection are indicated by a blue chevron symbol (>).

- 2 To see the connection cue, point to the chevron symbol (>).
- 3 Click the cue to connect the blocks with a line and an arrow that indicates the direction of signal flow.

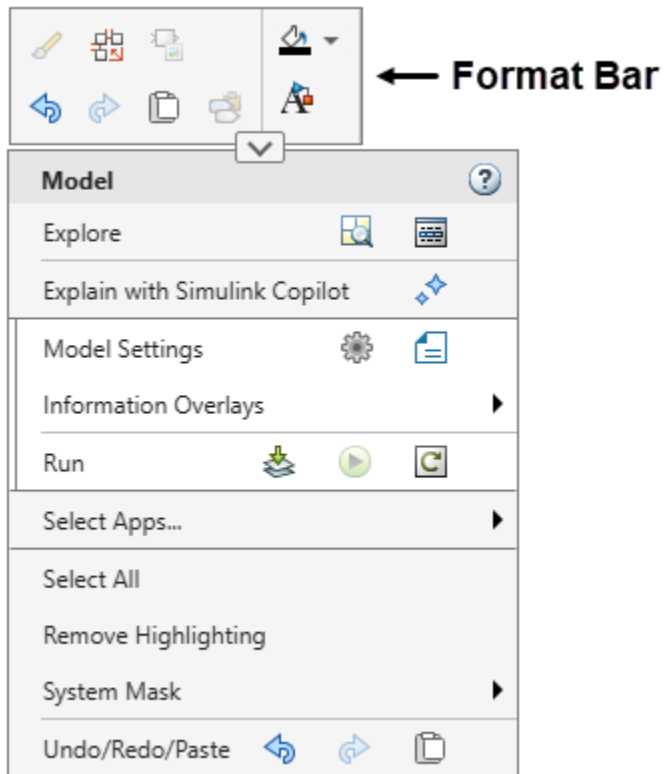
For more information about how to connect blocks, see “Connect Blocks”.



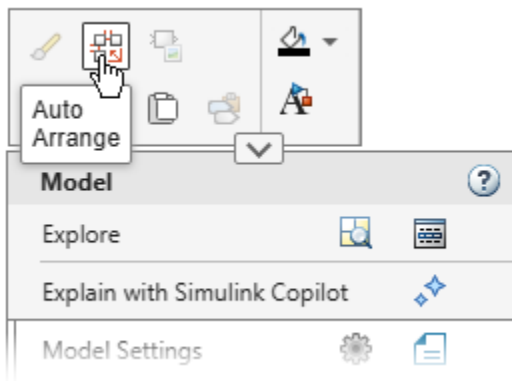
Once you have connected the blocks, line up the Pulse Generator, Gain, and Second-Order Integrator blocks by dragging each block. To resize a block, drag a corner of the block.

For large models, instead of dragging the individual blocks, you can improve the model layout using auto-arrange. Right-click the model canvas. The **Top Model** context menu appears. In Simulink context menus, formatting options such as changing colors or fonts, or auto-arranging the model, are located in the format bar. To expand the format bar, at the top of the menu, click the arrow .

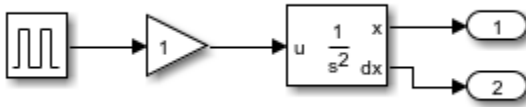
Then, click the Auto Arrange button .



Tip To view a tooltip that explains what action you can take by pushing a context menu button, pause your pointer on the button icon.



Auto-arrange aligns the blocks and straightens the signal lines.



Edit Block Parameter Values

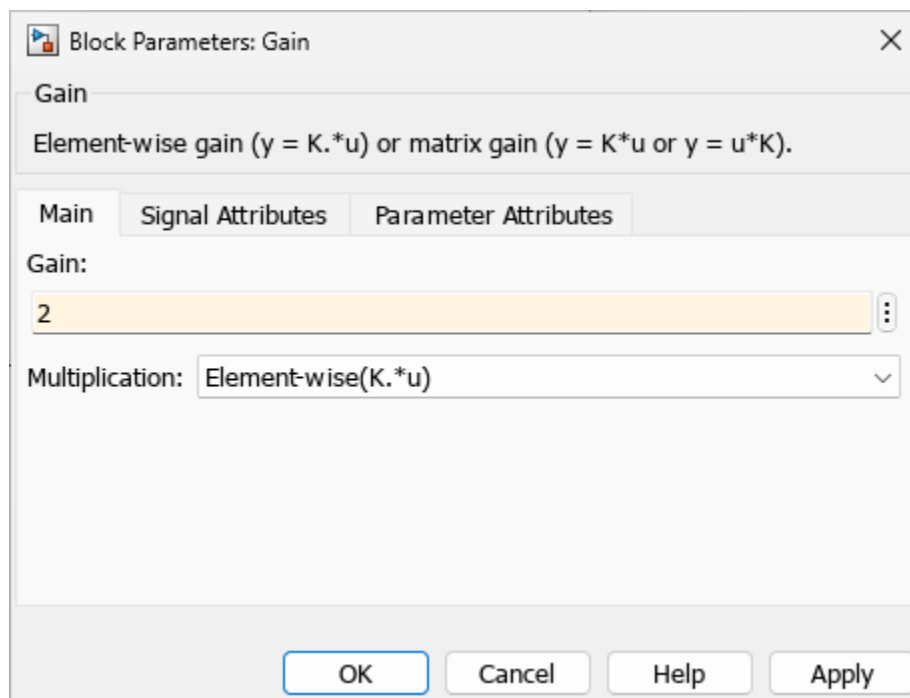
Blocks have parameter values that you can modify. To find out which parameters you can modify for a block and what type of values the parameters take, open the block documentation. Right-click the block, and in the upper right corner of the context menu, click the Open help documentation button



For some blocks, such as Constant blocks or Gain blocks, you can change a parameter value directly on the face of the block. Change the gain value of the Gain block in the example model to 2. Select the block, click the value on the block, enter the new value, and then press **Enter**.

You can also change the gain value in the Block Parameters dialog box. To open the Block Parameters dialog box, double-click the block. Alternatively, right-click the block, and then click the Block

Parameters button . In the dialog box that opens, change the **Gain** value to 2 and press **Enter**.



A third option is to use the Property Inspector. Select the Gain block. To open the Property Inspector, press **Ctrl+Shift+I** (on macOS, press **command+option+O**). Alternatively, right-click the block, and

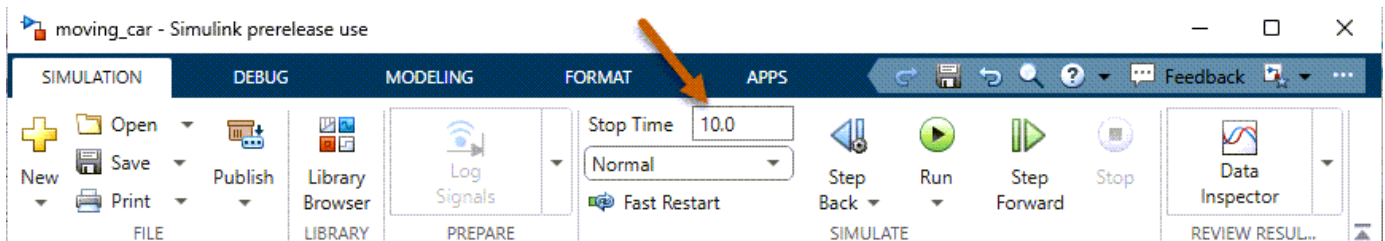
then click the Property Inspector button . In the Property Inspector, on the **Parameters** tab, change the **Gain** value to 2.

To change parameter values not displayed on the block icon, use the Block Parameters dialog box or the Property Inspector. If you do not see the name of a parameter whose value you want to change in the Block Parameters dialog box, check the Property Inspector, and vice versa.

Run Simulation

Specify the stop time for the simulation. Then, simulate the model.

- 1 On the **Simulation** tab, set the simulation stop time. In the Simulink Toolstrip, on the **Simulation** tab, enter the value in the **Stop Time** text box.



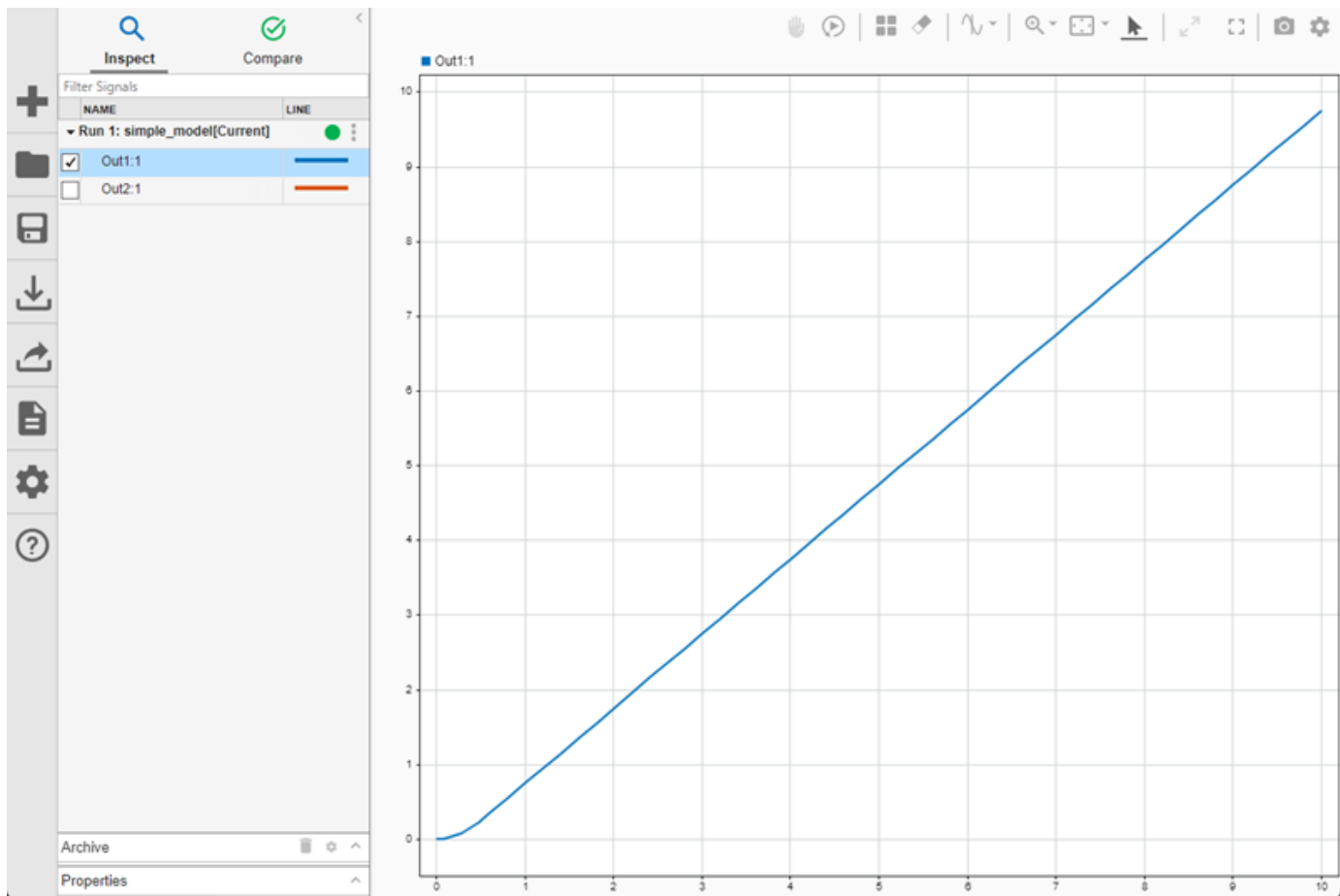
The default stop time of 10.0 is appropriate for this model. This time value has no units. The time unit in a Simulink simulation depends on how the equations are constructed. This example simulates the simplified motion of a car for 10 seconds, but other models could have time units in milliseconds or years.

- 2 To simulate the model, press **Ctrl+T** (on macOS, press **command+T**). Alternatively, in the toolstrip, on the **Simulation** tab, click **Run** .

View Simulation Data

To view simulation results in the Simulation Data Inspector, right-click one of the signal lines, and then click the View in Data Inspector button .

To plot data in the Simulation Data Inspector, select signals from the checklist on the left. For example, to plot the position of the car, select the signal named Out1:1.



Refine Model

You can refine a model by changing the block parameters, adding new blocks, making new connections, and annotating signal lines.

Change Block Parameters

This example models a proximity sensor based on an existing motion model named `moving_car`.

In this scenario, a digital sensor measures the distance between the car and an obstacle 10 m (30 ft) away. The model outputs the sensor measurement and the position of the car, taking these conditions into consideration:

- The car comes to a hard stop when it reaches the obstacle.
- In the physical world, a sensor measures the distance imprecisely, causing random numerical errors.
- A digital sensor operates at fixed time intervals.

Open the `moving_car` model.

```
open_system("moving_car.slx");
```

You first need to model the hard stop when the car position reaches 10. The Integrator, Second-Order block has a parameter for that purpose.

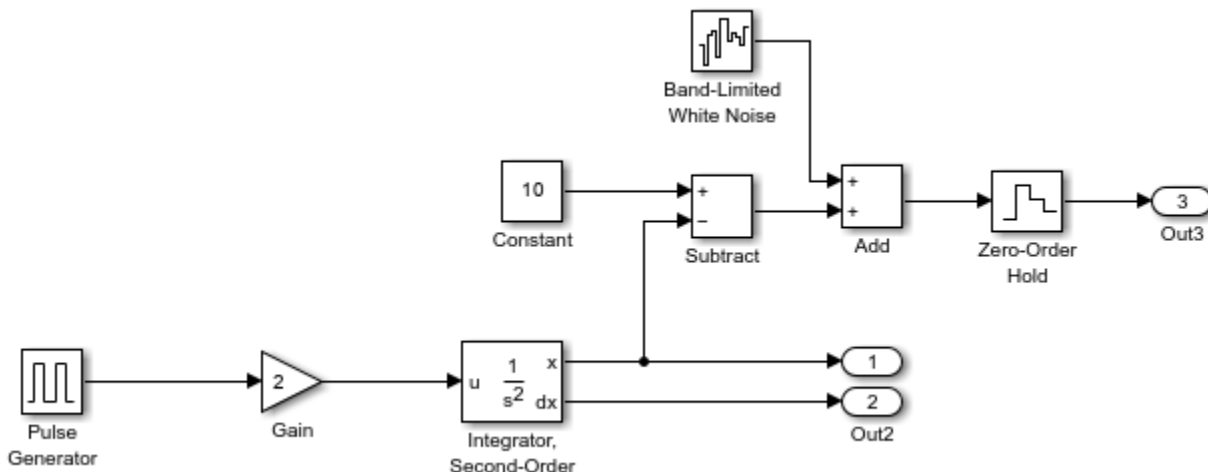
- 1 Double-click the Integrator, Second-Order block. The Block Parameters dialog box appears.
- 2 Select **Limit x** and enter 10 for **Upper limit x**. The background color for the parameter changes to indicate a modification that is not applied to the model. Click **OK** to apply the changes and close the dialog box.

Add New Blocks and Connections

Modify the model to add a sensor that measures the distance from the obstacle. Expand the model window to accommodate the new blocks as necessary.

- To find the distance between the vehicle and obstacle position, add a Constant block from the Sources library and set the value of block to 10. To find the distance between the obstacle position and the vehicle position, add the Subtract block from the Math Operations library.
- To simulate the imperfect measurements of a real sensor, add noise to the model by using the Band-Limited White Noise block from the Sources library. Double-click the block to set the **Noise power** parameter to 0.001. Add the noise to the distance measurement by using an Add block from the Math Operations library.
- In Simulink, sampling of a signal at a given interval requires a sample and hold. Add the Zero-Order Hold block from the Discrete library. Then, double-click the block to change the **Sample Time** parameter to 0.1.
- To log the sensor output, connect the Zero-Order Hold block to another Outport block.

Connect the new blocks. The output of the Second-Order Integrator block is already connected to another port. To create a branch in that signal, left-click the signal to highlight potential ports for connection, and click the appropriate port.



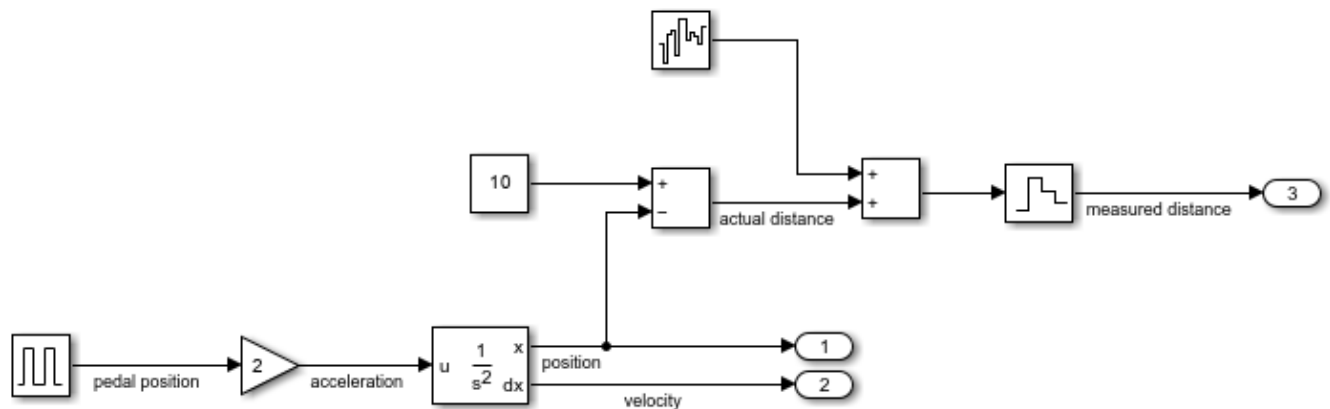
Annotate Signals

Add signal names to the model.

- 1 Double-click the signal and type the signal name.



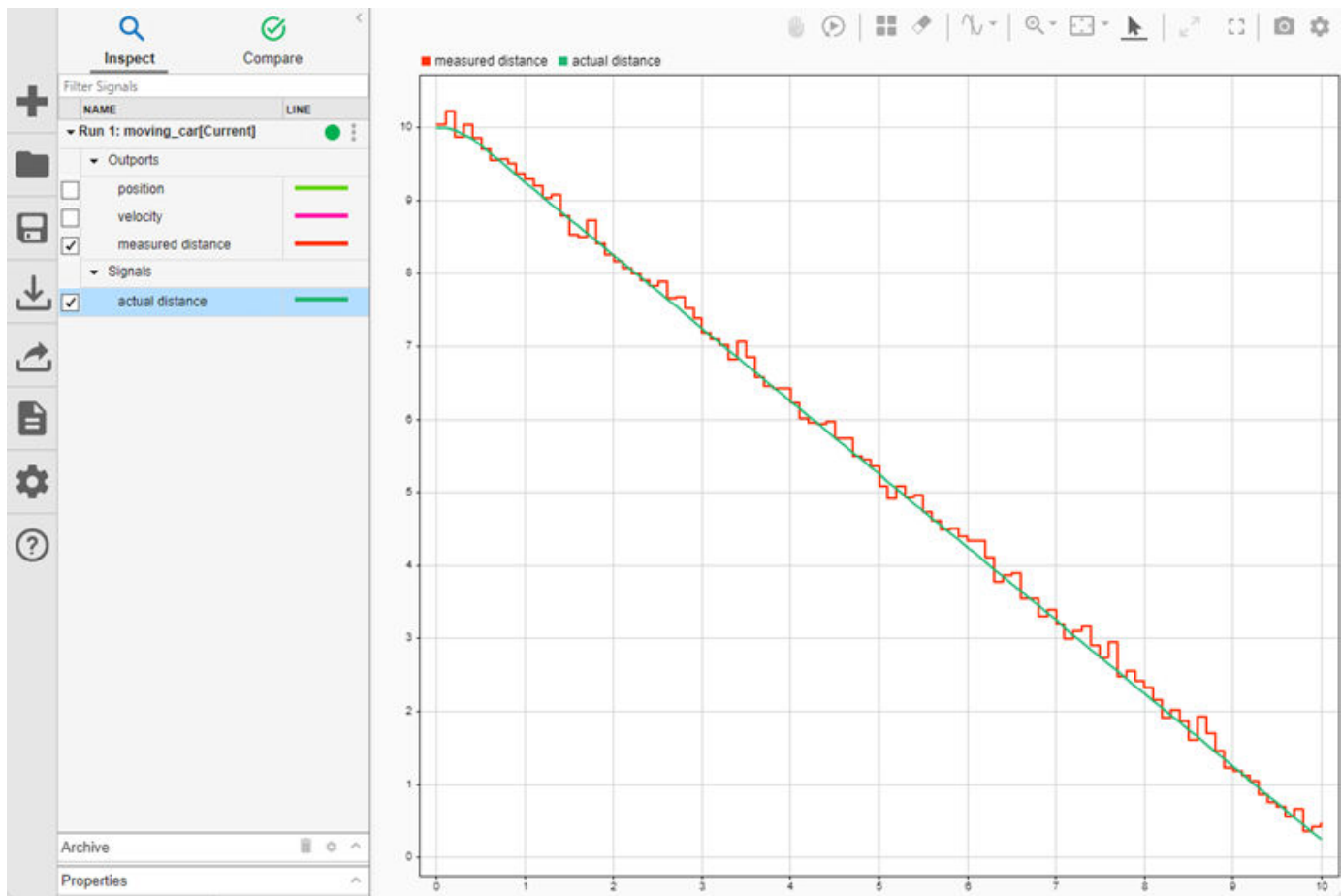
- 2 To finish, click away from the text box.
- 3 Repeat these steps to add the names as shown.



View Multiple Signals

Compare the `actual distance` signal with the `measured distance` signal. The `measured distance` signal is logged as output. To log the `actual distance` signal, you can mark it for signal logging. Right-click the signal line, and then click the Log Signals button . A logging badge  indicates that the signal is marked for logging.

Simulate the model. To view simulation results in the Simulation Data Inspector, right-click the signal line, and then click the View in Data Inspector button . Expand the **Outports** and **Signals** checklists. To plot both signals on the same time plot, select the `actual distance` and `measured distance` signals.



The plot shows that the measurement can deviate from the actual value by as much as 0.3 m. This information is useful when designing safety features, such as a collision warning.

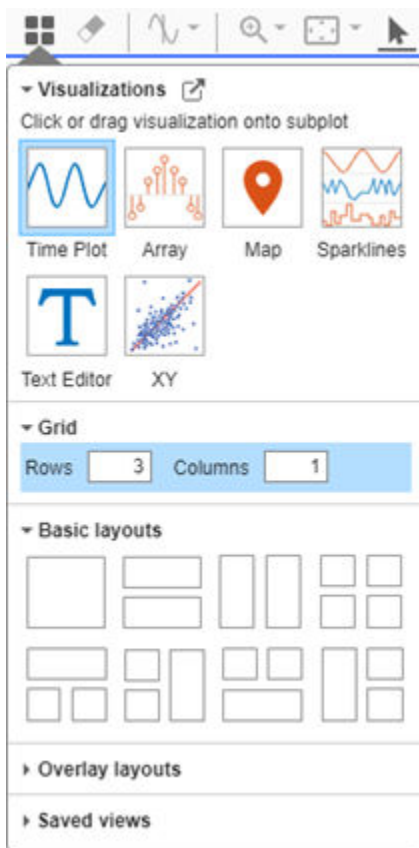
View Signals on Separate Subplots

You can also analyze results by viewing signals on separate subplots. For example, you can add subplots for the `pedal position` and `velocity` signals to view the relationship between the pedal position, the velocity of the car, and the distance between the car and the obstacle.

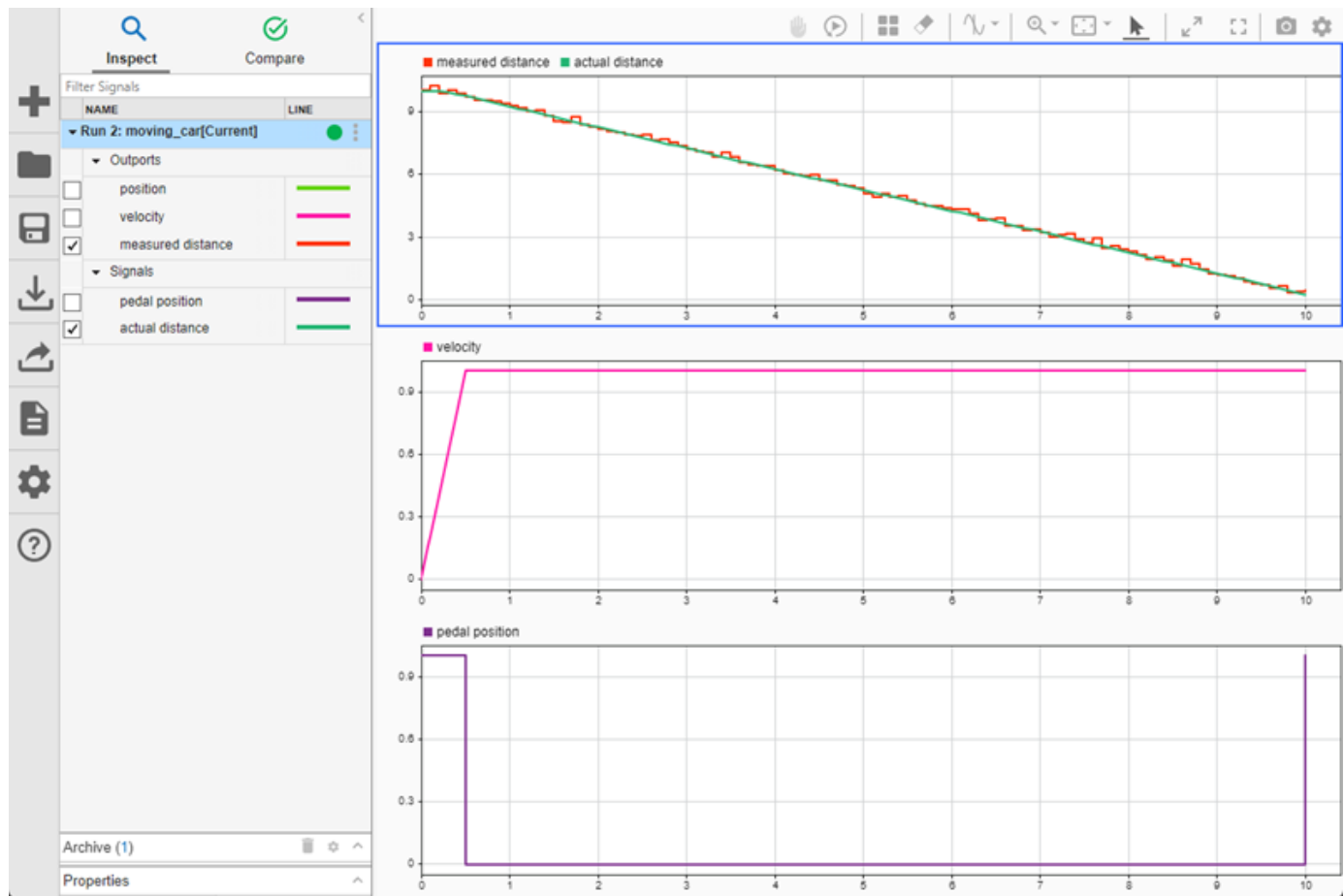
Mark the `pedal position` signal for signal logging. Right-click the `pedal position` signal line, and then click the Log Signals button .

Simulate the model. When the simulation finishes, in the Simulation Data Inspector, click

Visualizations and layouts . Then, create a 3-by-1 layout by specifying the number of rows and columns in the grid.

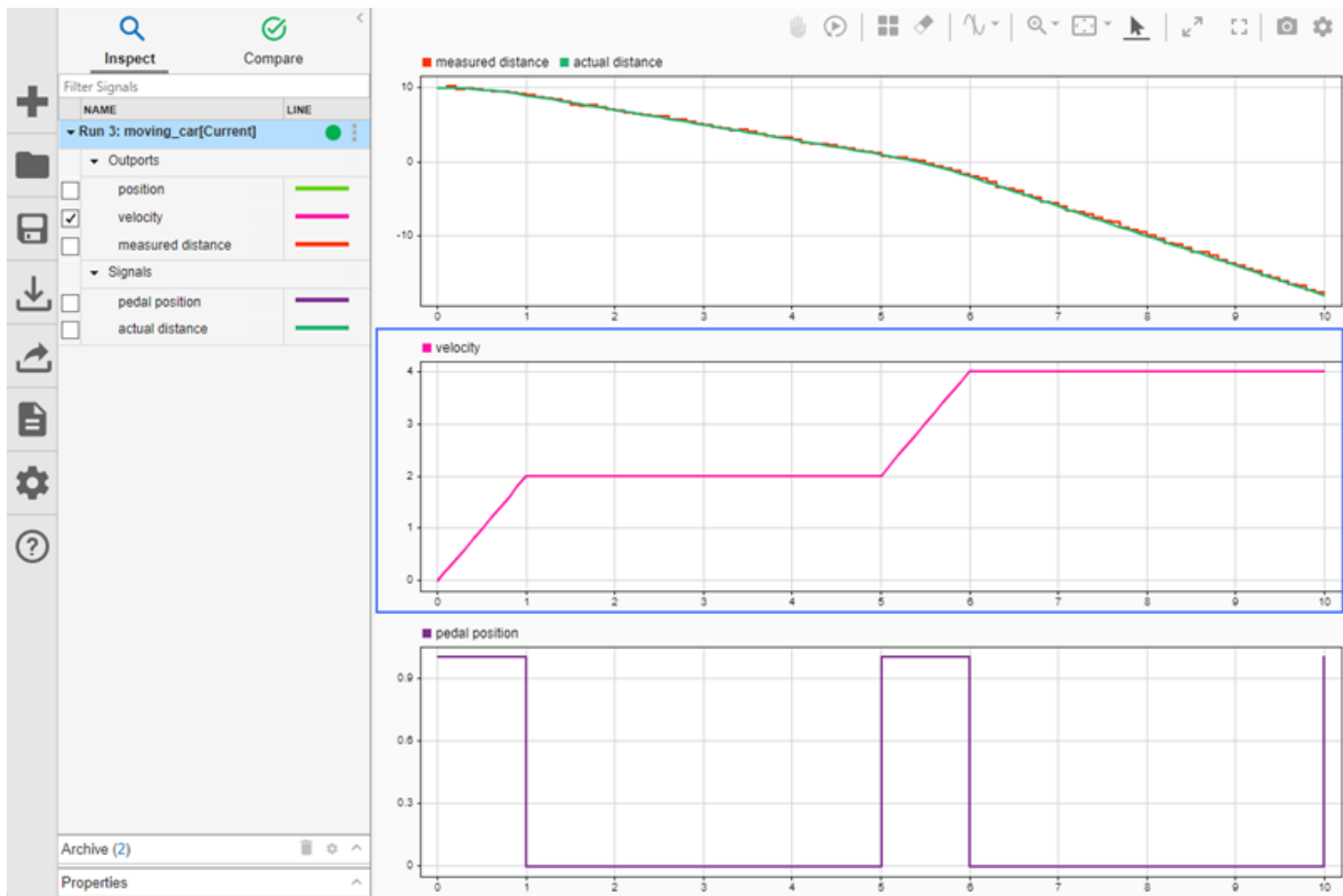


Add the velocity signal to the middle subplot and the pedal position signal to the bottom subplot. To add a signal to a subplot, select the subplot, and then select the signal from the signal table.



Visualizing the data on three subplots lets you to see the how pressing the gas pedal affects the velocity of the car and its distance from the obstacle. To explore this further, you can change the behavior of the gas pedal by adjusting the parameters of the Pulse Generator block. To open the Block Parameters dialog box for the Pulse Generator block, double-click the block. For example, model pressing the gas pedal for one second twice by setting **Period** to 5 and **Pulse Width** to 20.

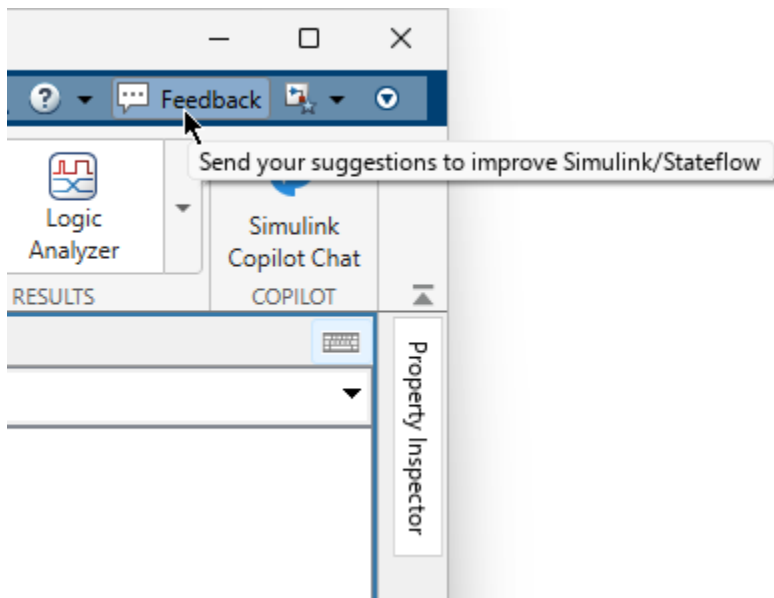
Simulate the model. In the Simulation Data Inspector, press the space bar to fit the signals to view.



In the Simulation Data Inspector, you can further inspect the data by customizing the plot and signal appearance, zooming and panning, and adding data cursors. For more information, see “Create Plots Using Simulation Data Inspector”.

Provide Feedback on the Software

To provide suggestions for improving Simulink and Stateflow®, in the quick access bar located in the upper right of the model window, click the **Feedback** button.



See Also

Blocks

Pulse Generator | Gain | Second-Order Integrator | Sum | Constant | Zero-Order Hold | Band-Limited White Noise

Tools

Simulation Data Inspector | Simulink Editor | Library Browser

Related Examples

- “Model and Validate a System” on page 1-13
- “Create Plots Using Simulation Data Inspector”

Navigate a Simulink Model

Explore Model Hierarchy

In this section...
"View Model Hierarchy" on page 4-2
"View Signal Attributes" on page 4-4
"Trace a Signal" on page 4-5

Simulink models can be organized into hierarchical components. In a hierarchical model, you can choose to view the system at a high level, or navigate down the model hierarchy to see increasing levels of model detail.

View Model Hierarchy

To start, open the `smart_braking` model.

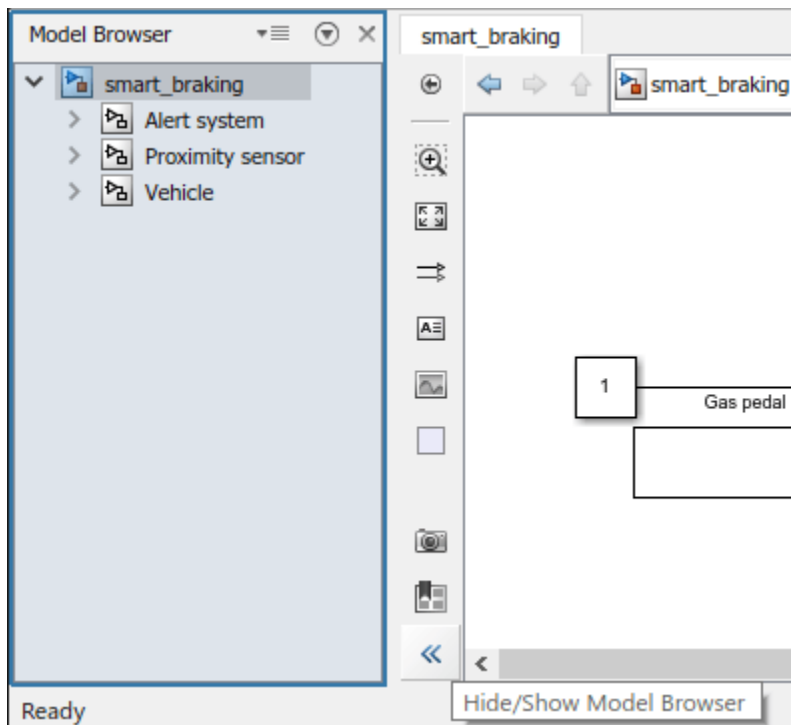
In the model:

- A vehicle moves as the gas pedal is pressed.
- A proximity sensor measures the distance between the vehicle and an obstacle.
- An alert system generates an alarm based on that proximity.
- The alarm automatically controls the brake to prevent a collision.

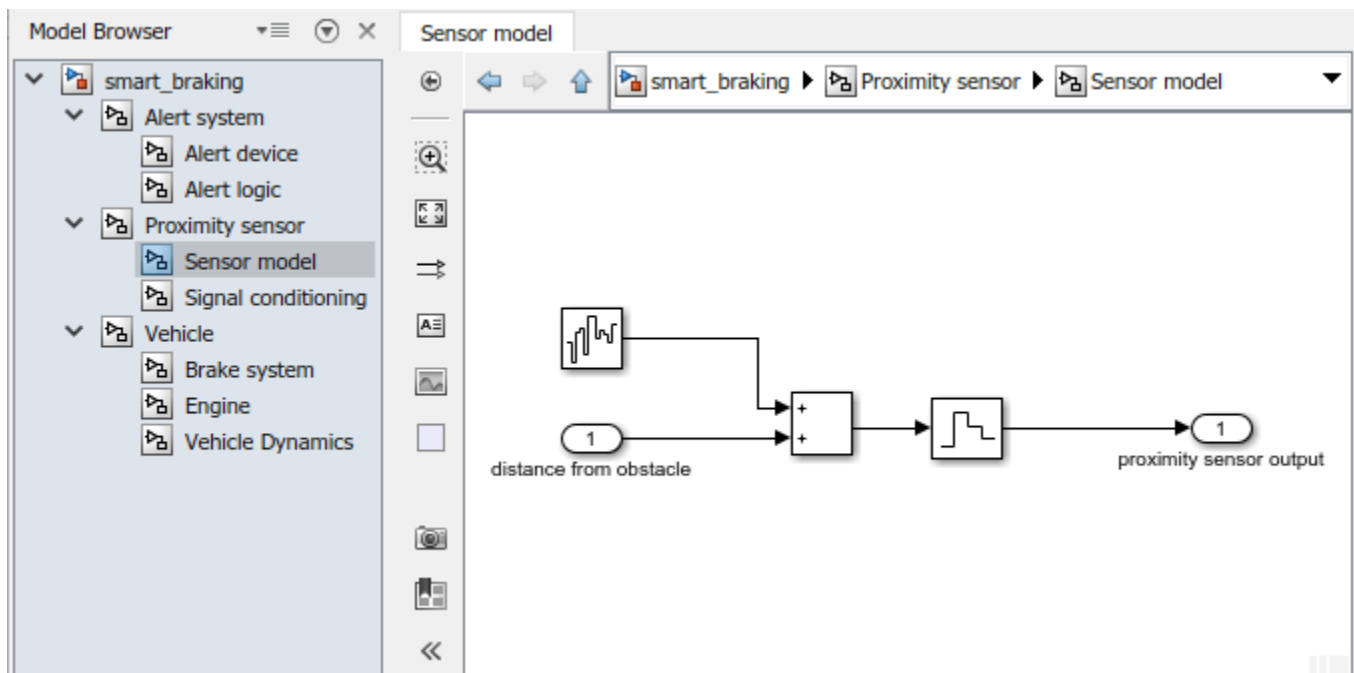
When you build a model, you connect blocks together to model complex components that represent system dynamics. In this model, Vehicle, Proximity sensor, and Alert system are all complex components with multiple blocks that exist in a hierarchy of subsystems. To view the contents of a subsystem, double-click the subsystem.

To view a representation of the complete model hierarchy, open the Model Browser.

- 1 Vertically expand the model window until the Hide/Show Model Browser button is visible in the lower left corner of the Simulink Editor.
- 2 Click the Hide/Show Model Browser button.



The Model Browser shows that all subsystems you view at the top level have subsystems of their own. Expand each subsystem node to see the subsystems it contains. You can navigate through the hierarchy in the Model Browser. For example, expand the Proximity sensor node and then select the Sensor model subsystem.



The address bar shows which subsystem you are viewing. To open the subsystem in a separate window, right-click the subsystem and then click the Open In New Window button.

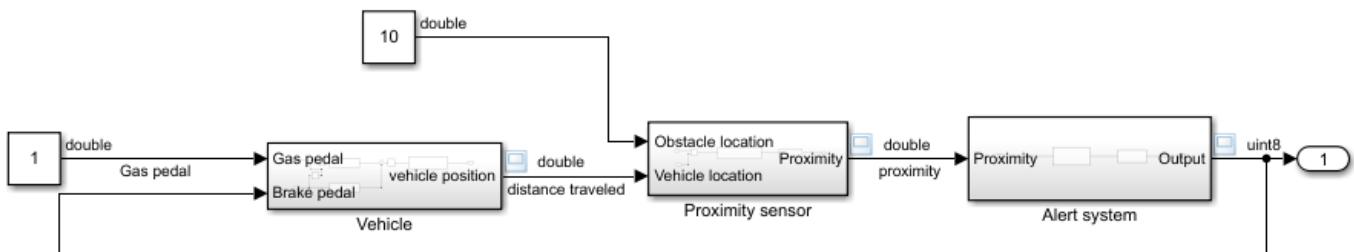
Every input or output port on a subsystem has a corresponding Inport or Outport block inside the subsystem. These blocks represent data transfer between a subsystem and its parent. When a system contains multiple input or output ports, the number on the Inport or Outport blocks indicates the position of the port on the subsystem interface.

View Signal Attributes

Signal lines in Simulink indicate data transfer from block to block. Signals have properties that correspond to their function in the model:

- Dimensions — Scalar, vector, or matrix
- Data type — String, double, unsigned integer, etc.
- Sample time — A fixed time interval at which the signal has an updated value (or 0 for continuous sampling)

To show the data type of all signals in a model, in the **Debug** tab, under **Information Overlays**, click **Base Data Types**.



The model displays data types along the signal lines. Most signals are double, except the output of the subsystem named **Alert system**. Double-click the subsystem to investigate.



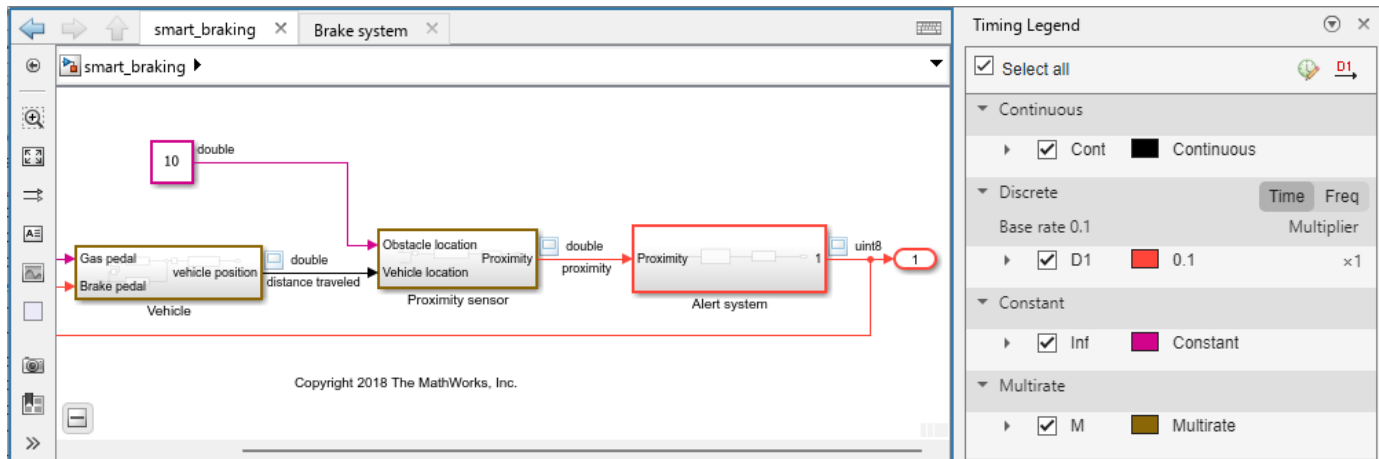
The data type labels in this subsystem show that data type change occurs in the subsystem named **Alert device**. Double-click the subsystem to investigate.



The alert device component converts the signal **Alert index** from a double to an integer. You can set the data type at sources, or use a Data Type Conversion block from the Signal Attributes library. Double, the default data type, provides the best numerical precision and is supported in all blocks.

The double data type also uses the most memory and computing power. Other numerical data types can be used to model embedded systems where memory and computing power are limited.

To show sample times, in the **Debug** tab, under **Information Overlays**, click **Colors** from the Sample Time section. The model updates to show different colors for each sample time in the model, along with a legend.



- A block or signal with continuous dynamics is black. Signals with continuous sample time update as often as the solver requires to satisfy specified tolerance values.
- A block or signal that is constant is magenta. They remain unchanged through simulation.
- A discrete block or signal that updates at the lowest fixed interval is red. Signals with discrete sample time update at a fixed interval. If the model contains components with different fixed sample times, each discrete sample time has a different color.
- Multirate subsystems, which contain a mix of discrete and continuous signals, are brown.

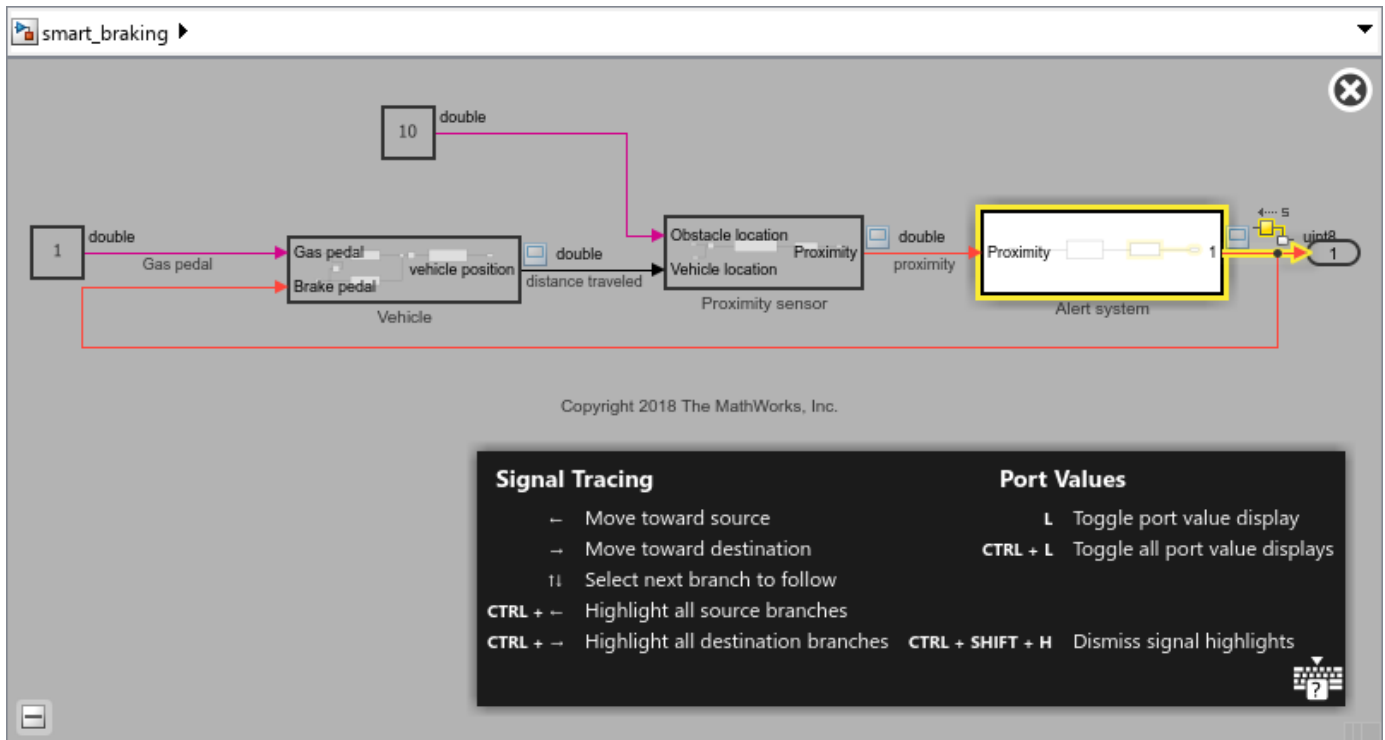
Trace a Signal

This model has a constant input and a discrete output. To determine where the sampling scheme changes, trace the output signal through its source blocks.

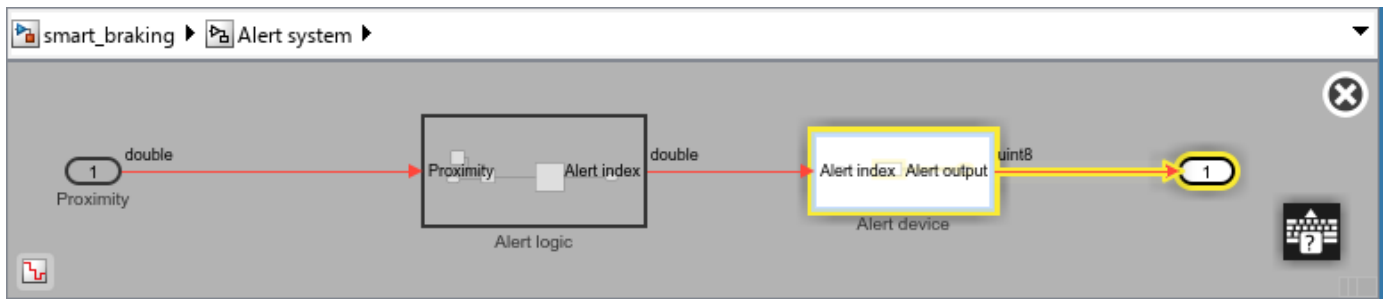
- 1 To open the Model Browser, vertically expand the model window until the Hide/Show Model Browser button  is visible in the lower left corner of the Simulink Editor. Then, click the button.
- 2 To start tracing the sources for the output signal, select the signal line connected to the output port of the subsystem named **Alert System**. Then, in the **Signal** tab, click **Trace to Source** .

The Simulink Editor enters signal tracing mode. In signal tracing mode, the model canvas becomes gray instead of white to help the traced path stand out.

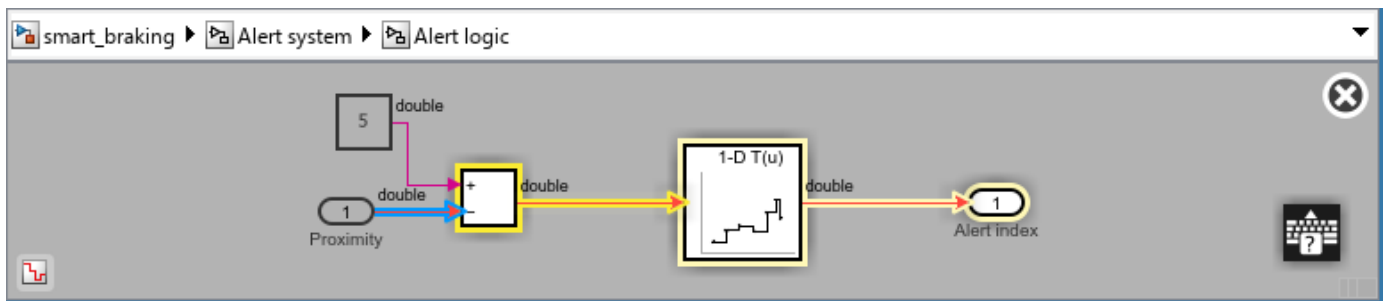
In the lower right, the hints panel shows keyboard shortcuts for actions you can take in signal tracing mode. To minimize or restore the hints panel, press **?** on your keyboard.



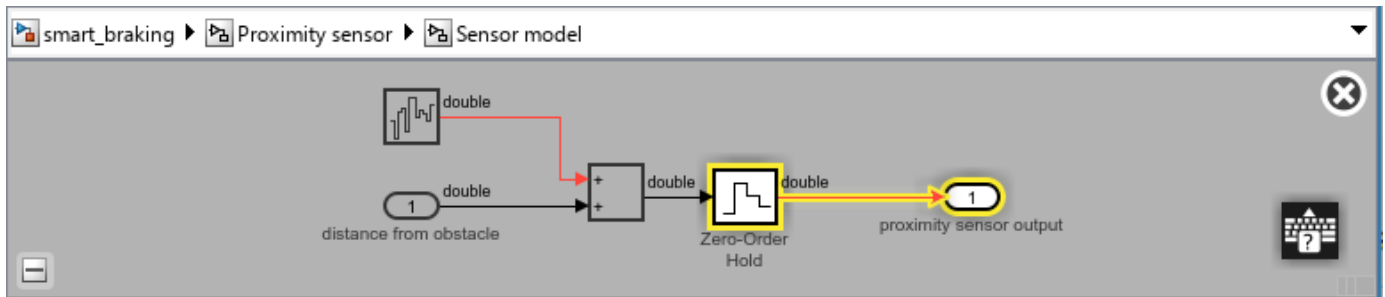
- 3 To continue tracing the sources for the output signal, press the left arrow key. The software navigates inside the subsystem named **Alert system** and highlights the next source block that affects the value of the output signal.



- 4 Keep pressing the left arrow key to trace the sources for the output signal until you reach the Subtract block inside the subsystem named **Alert logic**. When you reach the Subtract block in the signal path, you must choose a path to continue tracing because the Subtract block has two input ports. The software highlights the next segment to trace in blue to indicate the selected path. By default, the first input port is selected for continued tracing. Select the path for the minus input port by pressing the down arrow key.



5 To find the source of the discretization, continue pressing the left arrow.



The Zero-Order Hold block in the Sensor model subsystem converts the signal from continuous to discrete.

See Also

Related Examples

- “Create a Simple Model” on page 3-2
- “Model-Based Design with Simulink” on page 1-3

